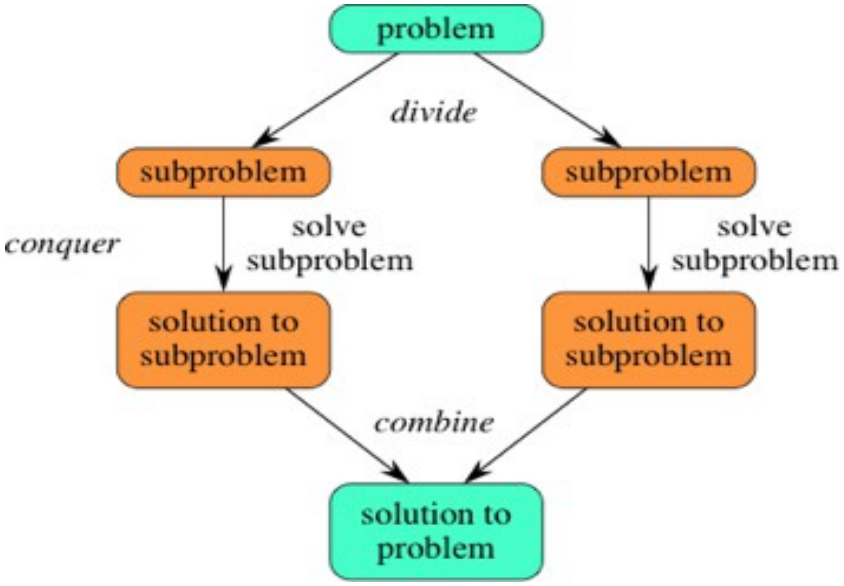
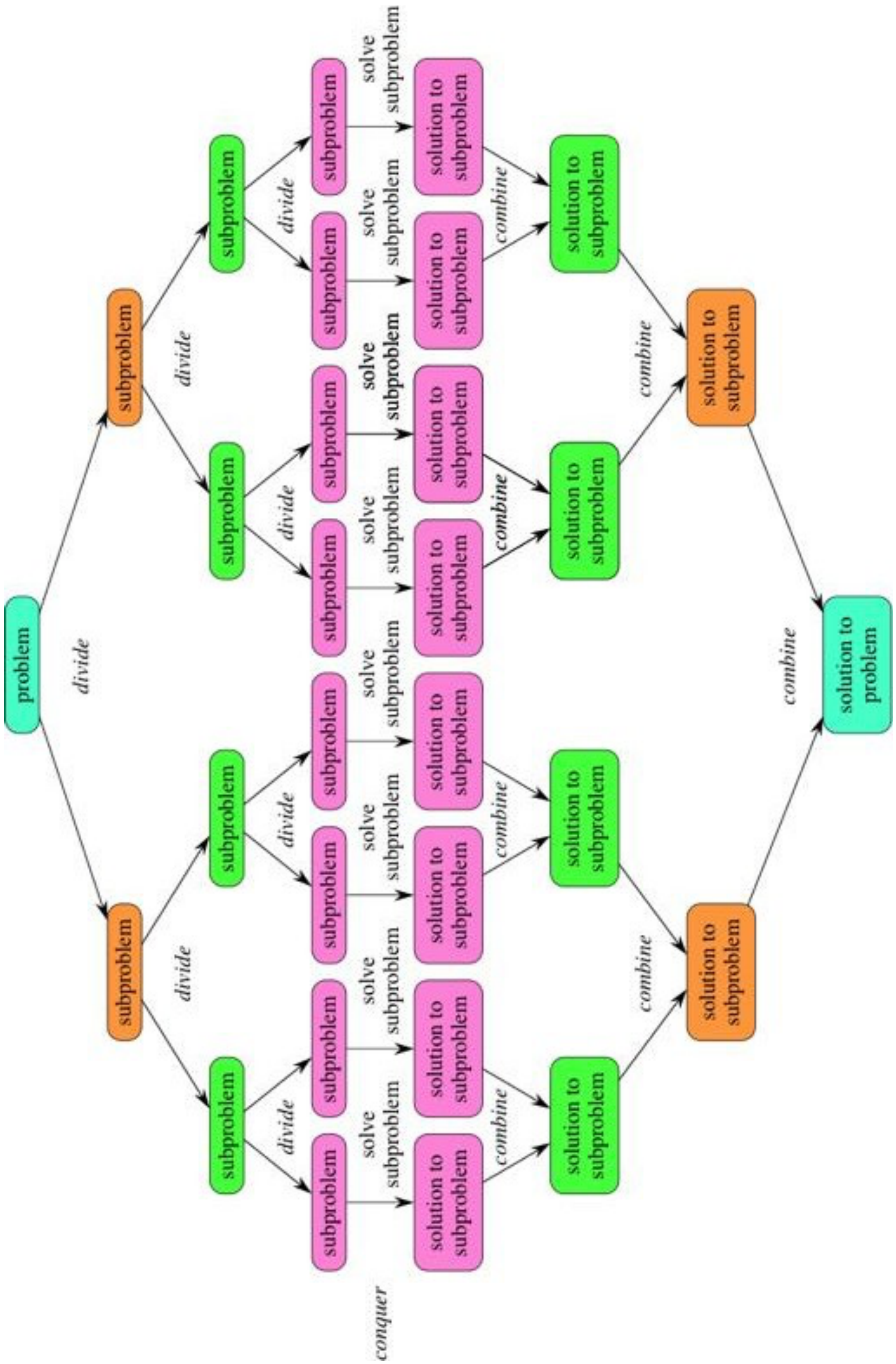
Unit-3: Divide and Conquer Algorithms

- Divide and Conquer is an algorithmic paradigm.
- This paradigm, divide-and-conquer, breaks a problem into subproblems that are similar to the original problem, recursively solves the subproblems, and finally combines the solutions to the subproblems to solve the original problem.
- Because divide-and-conquer solves subproblems recursively, each subproblem must be smaller than the original problem, and there must be a base case for subproblems.
- The divide-and-conquer paradigm involves three steps at each level of the recursion:
  1. **Divide** the problem into a number of subproblems that are smaller instances of the same problem.
  2. **Conquer** the subproblems by solving them recursively. If they are small enough, solve the subproblems as base cases.
  3. **Combine** the solutions to the subproblems into the solution for the original problem.
- The steps of a **divide-and-conquer** algorithm are **divide, conquer, combine**. Here's how to view one step, assuming that each divide step creates two subproblems (though some divide-and-conquer algorithms create more than two):



- If we expand out two more recursive steps, it looks like this:


problem
divide
subproblem
subproblem
divide
divide
subproblem
subproblem
subproblem
subproblem
divide
divide
divide
divide
subproblem
subproblem
subproblem
subproblem
subproblem
subproblem
subproblem
subproblem
solve subproblem
conquer
solution to subproblem
combine
solution to subproblem
combine
solution to subproblem
combine
solution to problem

- Because divide-and-conquer creates at least two subproblems, a divide-and-conquer algorithm makes multiple recursive calls.
- Following are some problems, which are solved using divide and conquer approach.
    - Finding the maximum and minimum items in a list of items
    - Strassen's matrix multiplication
    - Merge sort
    - Binary search

Pros and cons:

— Divide and conquer approach supports parallelism as sub-problems are independent. Hence, an algorithm, which is designed using this technique, can run on the multiprocessor system or in different machines simultaneously.

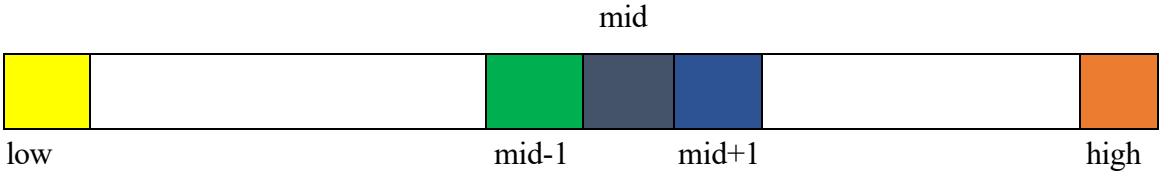
— In this approach, most of the algorithms are designed using recursion, hence memory management is very high. For recursive function stack is used, where function state needs to be stored.

Binary Search

- Binary search is an efficient algorithm for finding an item from a sorted list of items. It works by repeatedly dividing in half the portion of the list that could contain the item, until you've narrowed down the possible locations to just one.
- It requires the list to be in sorted order.
- In this method, to search an element you can compare it with the present element at the center (middle) of the list. If it matches, then the search is successful otherwise the list is divided into two halves: one from the 0th element to the middle element which is the center element (first half) another from the center element to the last element (which is the 2nd half) where all values are greater than the center element.
- The searching mechanism proceeds from either of the two halves depending upon whether the target element is greater or smaller than the central element. If the element is smaller than the central element, then searching is done in the first half, otherwise searching is done in the second half.

Following are the steps of implementation that we will be following:

- Start with the middle element:
  - If the target value is equal to the middle element of the array, then return the index of the middle element.
  - If not, then compare the middle element with the target value,
    - If the target value is greater than the number in the middle index, then pick the elements to the right of the middle index, and repeat the above steps.
    - If the target value is less than the number in the middle index, then pick the elements to the left of the middle index, and repeat the above steps.
- When a match is found, return the index of the element matched.
- If no match is found, then return -1



Algorithm

```
BinarySearch(A,low,high, key)
{
        if(low= = high)
        {
                if(key = = A[low])
                        return low; //search successful
                else
                        return -1; // search unsuccessful
        }
        else
        {
                mid= (low + high) /2 ; //integer division
                if(key = = A[mid]
                        return  mid;
                else if (key < A[mid])
                        return  BinarySearch(A, low, mid-1, key) ;
                else
                        return  BinarySearch(A, mid+1, high, key) ;
        }
}
```

Example:

Given list A = {2,3,5,7,9,11,13,17,19,23}, key = 13

Function Call	low	high	low=high	Mid= (Low+high)/2	Comparison of A[mid] and key	Remarks
1	0	9	no	(0+9)2 = 4	A[4] is 9 9<13	Search on right half low = mid+1 = 4+1 =5 high = 9
2	5	9	no	(5+9)2 = 7	A[7] is 17 17 > 13	Search on left half low =5 high = mid-1 = 7-1 =6
3	5	6	no	(5+6)2 = 5	A[5] is 11 11 < 13	Search on right half low =mid+1 = 5+1 =6 high =6
4	6	6	yes	-	-	A[low] = A[6] = 13 =key So, the search is successful Return 6

Analysis

Time complexity

In 1st search n sized list reduces to n/2 size

In 2nd search n/2 sized list reduces to $n/2^2$ size

In 3rd search $n/2^2$ sized list reduces to $n/2^3$ size

Similarly,

In k^{th} search $n/2^{k-1}$ sized list reduces to $n/2^k$ size

Suppose in k^{th} search the list reduces to 1.

i.e. $n/2^k = 1$

$=> 2^k = n$

$=> k = \log_2 n$

Total steps required to search an item in list = $k = \log_2 n$

Hence, Time complexity for this algorithm = $O(\log_2 n)$

We can express the time complexity using recurrence relation as:

$T(n) = T(n/2) + O(1)$, $n>1$

$= 1$, $n= 1$

After solving this recurrence relation,

$T(n) = O(\log_2 n)$

Space Complexity

Call stack is used to perform recursion. There are at most $\log_2 n$ recursive calls. So the space complexity = $O(\log_2 n)$

Note: In iterative approach space complexity is O(1)

Max-Min Algorithm

- To find maximum and minimum element of a unsorted list of elements
- Here our problem is to find the minimum and maximum items in a set of n elements. We are exploring two methods here first one is iterative approach (Naïve method) and the next one uses divide and conquer strategy to solve the problem.

Naïve method

Naïve method is a basic method to solve any problem. In this method, the maximum and minimum number can be found separately.

We initialize both minimum and maximum element to the first element and then traverse the array, comparing each element and update minimum and maximum whenever necessary.

Pseudo Code

```
MaxMin(A,n)
{
   max = min = A[0];
   for(i = 1; i < n; i++)
   {
      if(A[i] > max)
           max = A[i];
      else if(A[i] < min)
           min = A[i];
   }
}
```

Analysis:

At every step of the loop, we are doing 2 comparisons in the worst case. Total no. of comparisons (in worst case) = 2*(n-1) = 2n – 2

The best case occurs when the elements are in increasing order with (n-1) comparisons and worst case occurs when elements are in decreasing order with 2(n-1) comparisons. For the average case A[i] > max is about half of the time so number of comparisons is 3n/2 – 1.

We can clearly conclude that the time complexity is O(n).

Time complexity = O(n),

Similarly, Space complexity = O(1)

Note: The number of comparisons can be reduced using the divide and conquer approach.

Divide and Conquer Approach

In this approach, the array is divided into two halves. Then using recursive approach maximum and minimum values in each halves are found. Later, return the maximum of two maxima of each half and the minimum of two minima of each half.

Main idea behind the algorithm is: if the number of elements is 1 or 2 then max and min are obtained trivially. Otherwise split problem into approximately equal part and solved recursively.

```
MaxMin(A,low,high)
{
   if(low = = high)
      max = min = A[low];
   else if(low== high-1)
   {
      if(A[low] < A[high])
      {
           max = A[high];
           min = A[low];
      }
      else
      {
           max = A[low];
           min = A[high];
      }
   }
   else
   {
      //Divide the problems
           mid = (low+ high)/2; //integer division
      //solve the subproblems
           {max,min}=MaxMin(A,low,mid);
           {max1,min1}= MaxMin(A, mid +1,high);
       //Combine the solutions
           if(max1 > max)
                max = max1;
           if(min1 < min)
                min = min1;
   }
}
```

Analysis:

- We can give recurrence relation as below for MaxMin algorithm in terms of number of comparisons.

  $T(n) = 2T(n / 2) + O(1)$, if $n>2$

  $T(n) = 1$, if $n \leq 2$

  Solving the recurrence relation we get time complexity O(n).

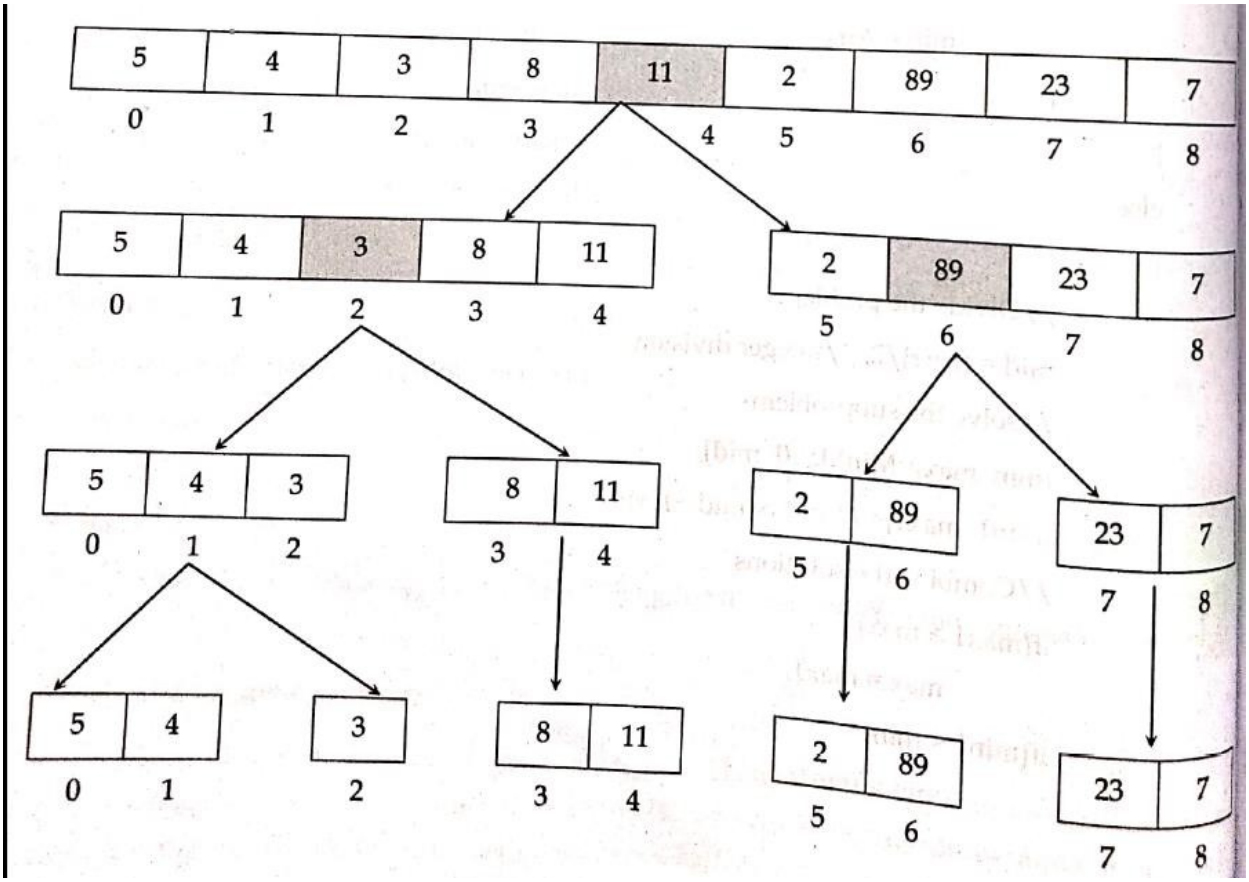
```
T(n) = 2 T(n/2) + 2
T(2) = 1
T(1) = 0
We can solve this recurrence relation
If n is a power of 2
     T(n) = 3n/2 - 2
```

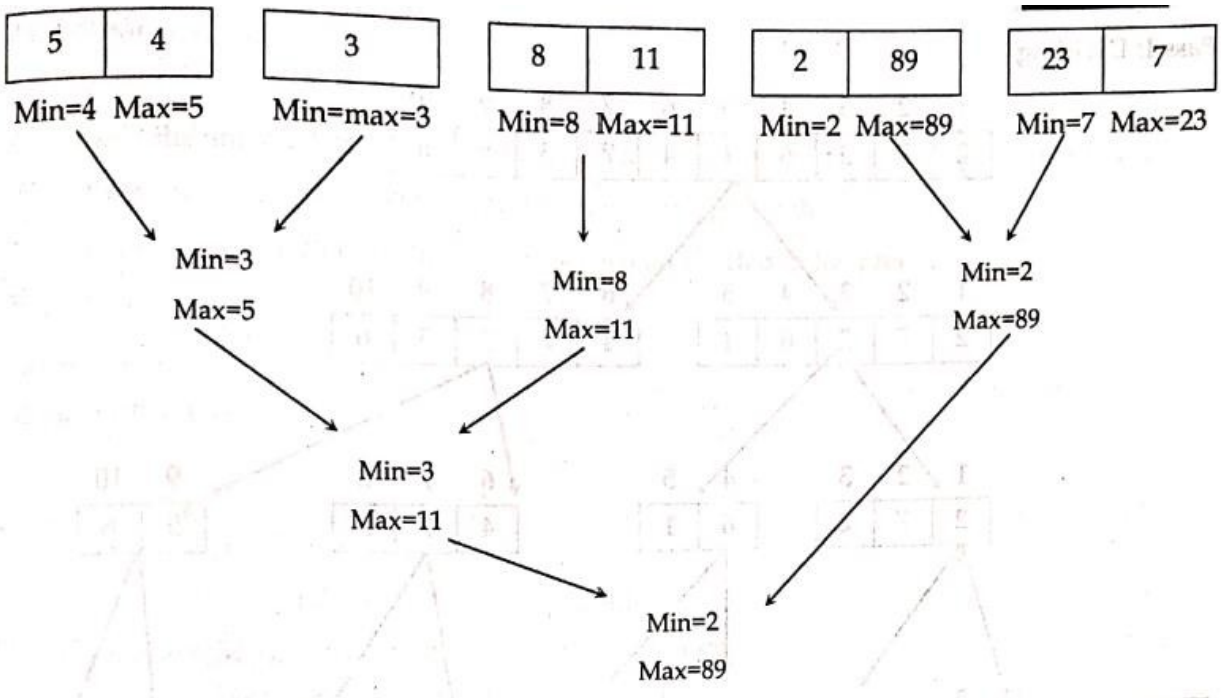
  Note: If n is a power of 2, the algorithm needs exactly 3n/2–2 comparisons to find min and max. If it's not a power of 2, it will take a few more (not significant).

- Space complexity = O(logn) (For recursion call stack)

Example: A ={5,4,3,8,11,2,89,23,7}

Tracing of MaxMin algorithm(divide and conquer approach)


5 4 3 8 11 2 89 23 7
0 1 2 3 4 5 6 7 8
5 4 3 8 11
0 1 2 3 4
2 89 23 7
5 6 7 8
5 4 3
0 1 2
8 11
3 4
2 89
5 6
23 7
7 8
5 4
0 1
3
2
8 11
3 4
2 89
5 6
23 7
7 8


5 4
Min=4 Max=5
3
Min=max=3
8 11
Min=8 Max=11
2 89
Min=2 Max=89
23 7
Min=7 Max=23
Min=3
Max=5
Min=8
Max=11
Min=2
Max=89
Min=3
Max=11
Min=2
Max=89

Quick sort

- Based on divide and conquer approach.
- Quick sort is also known as partition exchange sort.
- The main idea of Quick sort is partitioning an array about an element called pivot.
- An array A[ ] is partitioned about an pivot element A[i] if all the elements to the left of A[i] are less than or equal to A[i] and all the elements to the right of A[i] are greater than A[i].

  A[] ={5,9,6,8,10,14,11,16,19,15}

  Here array A is partitioned about A[4] =10

  But A is not partitioned about A[5] = 14
- The partitioning step of Quick Sort rearrange the array so that the pivot element is placed on it's proper position on the sorted list and all the elements to the left are smaller or equal to pivot and elements to the right are greater than pivot.
- There are many different versions of QuickSort that pick pivot in different ways.(first element, last element, median element, random element, etc can be chosen as pivot)

Outline of Quick Sort Algorithm (pivot = first element)

1. Take the first element of array as pivot
2. Set Left marker 'L' at the beginning of array and right Marker 'R' at the end of the array.
3. While L< R, Repeat
   - Increment L (move right) until the element at index position is less than or equal to pivot element
   - Decrement R (move left) until the element at index position is greater than pivot element
   - If (L< R)
     - Swap (A[L], A[R]
4. Swap pivot with A[R]
5. Repeat 1-4 on left part and right part of pivot till the entire list is sorted.

Divide, Conquer and Combine Steps in Quick Sort

- **Divide**

  Partition the array A[0…r] into 2 subarrays A[0..m] and A[m+1..r], such that each element of A[0..m] is smaller than or equal to each element in A[m+1..r]. Need to find index *p* to partition the array.

- **Conquer**

  Recursively sort two sub-arrays using partitioning

- **Combine**

  Two sub-arrays are sorted already. The result of combination is also sorted.

Pseudo code of Quick Sort Algorithm and Partition

```
QuickSort(A,p,q){
    if (p<q){
            j = Partition(A,p,q);
            QuickSort(A,p,j-1);
            QuickSort(A,j+1,q);
        }
    }

Partition(A, i, j){
    L = i;
    R= j;
    pivot = A[i];
    while (L< R){
        while(A[L] < = pivot && L< = j){
            L++ ;
        }
        while(A[R] > pivot){
            R--;
```

```
        }
        if(L< R){
            swap (A[L], A[R]);
        }
    }
    swap (pivot, A[R]);
    return R;
}
```

Example: Sorting following array A, using Quick Sort

25	10	30	15	20	28

- Here, pivot = 25 , L and R are pointing at beginning and end of the array

0	1	2	3	4	5
25	10	30	15	20	28
L					R

- Now Since L<R and L< = j(5), Increase L until the element is < = 25 and decrease R until element is > 25

0	1	2	3	4	5
25	10	30	15	20	28
		L		R	

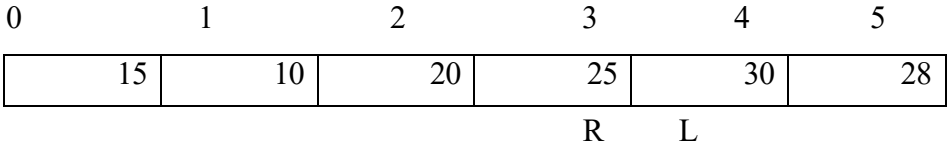
Here, L<R so, swap (A[L], A[R]) i.e. swap 30 and 20

0	1	2	3	4	5
25	10	20	15	30	28
		L		R	

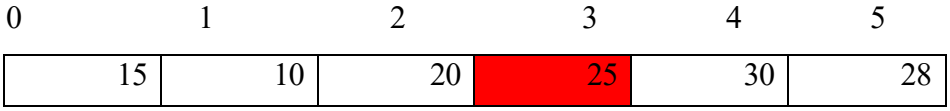
- Again increase L and Decrease R

0	1	2	3	4	5
25	10	20	15	30	28
			R	L	

Here, L>=R so, swap pivot and A[R]

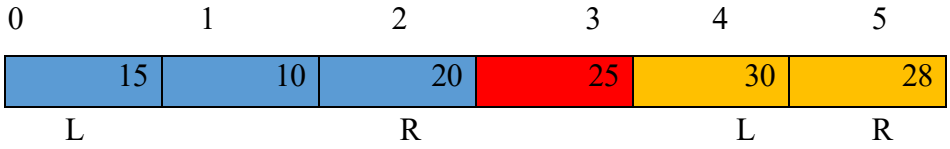


Partition is complete around pivot 25.

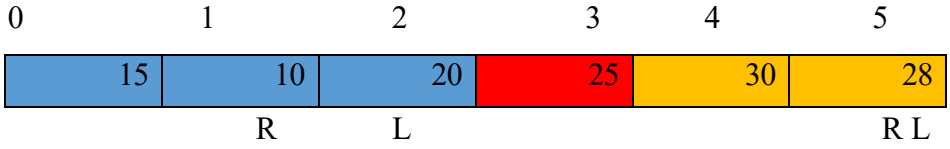


Left_sub_array[ ] = {15,10,20} and Right sub_array = {30,28 }

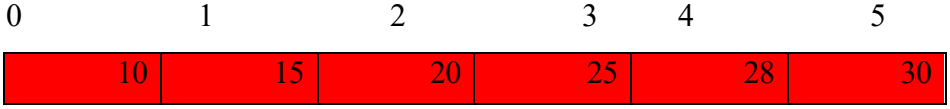
- The process is repeated on subarrays also.
- For the left subarray pivot = 15, L=0, R = 20 similarly, for right subarray pivot =30, L=4,R=5



- Now Increasing L and decreasing R on both subarrays .(In right subarray, R is not decreased since A[R] < pivot)



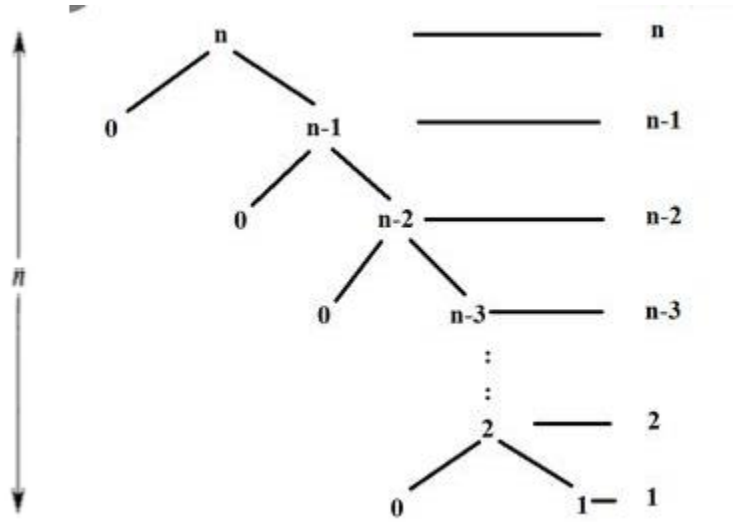
- In left subarray Swap A[R] and pivot i.e.15. In right subarray also swap A[R] and pivot i.e. 30. As a result, left subarray is partitioned around 15 and right subarray is partitioned around 30. There are single elements in both sides of 15, which therefore are sorted. We can clearly see that 28 is single element in left of 30 which is also sorted. Hence all elements now are sorted.



Analysis

Time Complexity

Worst Case



— When the array is already in sorted order (ascending order)

— The first element (smallest in the list) is taken as pivot and total n comparison are performed for partitioning and then the pivot position remains in its own position. The left sub-list contains zero element and right sub-list size is n-1

— Again the sub-list of size n-1 is partitioned by same process where the first element remains on its own position as pivot after n-1 comparisons and the list size reduces to n-2 and so on.

— So, $T(n) = n + (n-1) + (n-2) + \ldots + 2 + 1$

$= n(n+1)/2$

$= n^2/2 + n/2$

Hence, Time complexity = $O(n^2)$

We can express the time complexity in worst case using recurrence relation as:

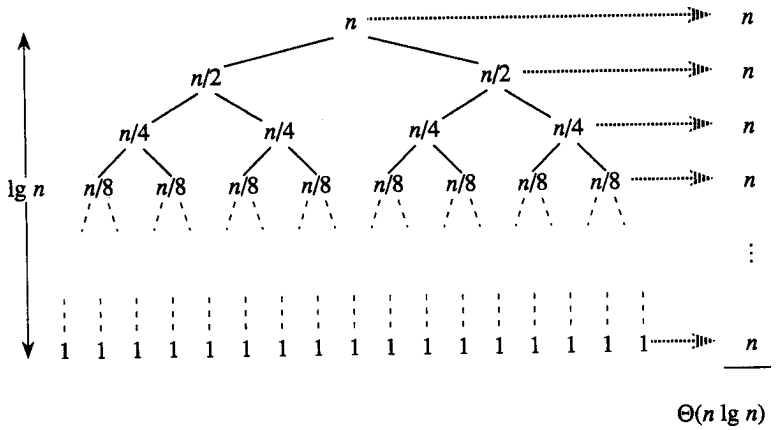
$T(n) = T(n-1) + O(n)$, $n>1$

$= 1$, $n= 1$

After solving this recurrence relation,

$T(n) = O(n^2)$

Best Case



- When the input array is in random order and each partitioning step divides the list into two sub-list of approximate size n/2.
- Divides the array into two partitions of equal size.
- In the first partitioning step. One element remains on its own sorted position and two sub-list of n/2 size are created and there are n comparisons for it.
- In successive partitioning, for sub-lists of size n/2, there are n/2 comparisons of each and the sub-list of size n/2 are divided into 4 sub-lists of size n/4 each.
- Suppose that the list size $n = 2^k$ then at k^{th} partitioning step, there are at most n sub lists of size 1
- So, $T(n) = n*1 + (n/2)*2 +(n/4)*4 +(n/8)*8 + \ldots + (n/2^k) * 2^k$

  $= n + n + n +n + \ldots + n$ [k times)

  $= kn$

  Since $2^k = n$

  $k = \lg n$

So,

T(n) = lgn * n

So, Time complexity = O(nlgn)

We can express the time complexity in best case using recurrence relation as:

T(n) = 2T(n/2) + O(n) , n>1

= 1 , n= 1

After solving this recurrence relation,

T(n) = O(nlgn)

Average case

- When the input array is random such that partitioning position at each step is not always same position of list i.e sometimes at center and sometimes at other positions
- All permutations of the input numbers are equally likely. On a random input array, we will have a mix of well balanced and unbalanced splits.
- In this case the time complexity function will be linearthmic with some additional linear or constant factor i,e T(n) = c1* nlgn + c2*n + c3
- So, Time complexity = O(nlgn)

Suppose the partition has balanced and unbalanced alternate: Balanced, Unbalanced,Balanced, ….

Representing in recurrence relation,

B(n)= 2UB(n/2) + Θ(n) //Balanced

UB(n)= B(n –1) + Θ(n) //Unbalanced

Solving:

B(n)= 2(B(n/2 –1) + Θ(n/2)) + Θ(n)

= 2B(n/2 –1) + Θ(n)

= Θ(nlogn)

Hence T(n) = O(nlogn)

Space Complexity:

Stack is used to perform recursion.

Best Case: There are at most $\log_2 n$ recursive call records in stack. So the space complexity = $O(\log_2 n)$

Worst Case: There are at most n recursive call records in stack. So the space complexity = $O(n)$

Randomized Quick Sort

- In the normal partitioning of the quick sort, we choose the pivot element either the first element or the last element of the list. For random list it works fine but for the list which is already sorted the complexity of normal partitioning quick sort is large.
- So, to improve the performance of quick sort for already sorted or nearly sorted list, we can use the randomized partitioning so that at each step the pivot element is taken at random position. So, it guarantees that both sub-lists are not empty at most of the steps of portioning.
- The basic idea of randomized partitioning is:
    - Choose pivot from a random index position
    - Exchange the first element with that pivot element
    - Apply the normal partitioning taking 1st element as pivot
- The algorithm is called randomized if its behavior depends on input as well as random value generated by random number generator. The beauty of the randomized algorithm is that no particular input can produce worst-case behavior of an algorithm
- Running time is independent of the input order. No assumptions need to be made about the input distribution. No specific input elicits the worst-case behavior. The worst case is determined only by the output of a random-number generator. Randomization cannot eliminate the worst-case but it can make it less likely!

Algorithm:

```
RandmizedQuickSort(A,p,q)
{
    if(p<q)
    {
        j = RandmizedPartition(A,p,q);
        RandmizedQuickSort(A,p,j-1);
```

```
        RandmizedQuickSort(A,j+1,q);
    }
}
```

```
RandmizedPartition(A,m,p)
{
    i = random(m,p); //generates random number between m and p including both.
    swap(A[m],A[i]);
    return Partition(A,m,p);// normal partitioning Algorithm
}
```

Time Complexity

Worst Case:

T(n) = worst-case running time

$T(n) = \max_{1 \le q \le n-1} (T(q) + T(n-q)) + \Theta(n)$

Use substitution method to show that the running time of Quicksort is $O(n^2)$

Guess $T(n) = O(n^2)$

- Induction goal: $T(n) \le cn^2$
- Induction hypothesis: $T(k) \le ck^2$ for any $k < n$

Proof of induction goal:

$T(n) \le \max_{1 \le q \le n-1} (cq^2 + c(n-q)^2) + \Theta(n)$

$= c \cdot \max_{1 \le q \le n-1} (q^2 + (n-q)^2) + \Theta(n)$

The expression $q^2 + (n-q)^2$ achieves a maximum over the range $1 \le q \le n-1$ at one of the endpoints

$\max_{1 \le q \le n-1} (q^2 + (n - q)^2) = 1^2 + (n - 1)^2 = n^2 - 2(n - 1)$

$T(n) \le cn^2 - 2c(n - 1) + \Theta(n)$

$\leq cn^2$

Note: The purpose of randomization is to make the worst-case unlikely.

Average Case

To analyze average case, assume that all the input elements are distinct for simplicity. If we are to take care of duplicate elements also the complexity bound is same but it needs more intricate analysis. Consider the probability of choosing pivot from n elements is equally likely i.e. 1/n.

Now we give recurrence relation for the algorithm as

$$T(n) = 1/n \sum_{k=1}^{n-1} (T(k) + T(n-k)) + O(n)$$

For some k = 1,2, …, n-1, T(k) and T(n-k) is repeated two times

$$T(n) = 2/n \sum_{k=1}^{n-1} T(k) + O(n)$$

$$nT(n) = 2 \sum_{k=1}^{n-1} T(k) + O(n^2)$$

Similarly

$$(n-1)T(n-1) = 2 \sum_{k=1}^{n-2} T(k) + O(n-1)^2$$

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2n - 1$$

$$nT(n) - (n+1)T(n-1) = 2n-1$$

$$T(n)/(n+1) = T(n-1)/n + (2n+1)/n(n-1)$$

Let $An = T(n)/(n+1)$

$$\Rightarrow \quad A_n = A_{n-1} + (2n+1)/n(n-1)$$

$$\Rightarrow \quad An = \sum_{i=1}^{n} 2i - 1/\, i(i+1)$$

$$\Rightarrow \quad An \approx \sum_{i=1}^{n} 2i / i(i+1)$$

$\Rightarrow \quad An \approx 2\sum_{i=1}^{n} 1/(i+1)$

$\Rightarrow \quad An \approx 2\log n$

Since An = T(n) /(n+1)

T(n) = nlogn

Merge Sort

- Based on Divide and Conquer approach.
- Conceptually, a merge sort works as follows:
  - Divide the unsorted list into n sublists, each containing one element (a list of one element is considered sorted).
  - Repeatedly merge sublists to produce new sorted sublists until there is only one sublist remaining. This will be the sorted list.
- Merge sort is an efficient sorting technique which is based on divide and conquer paradigm.
- In this technique, an array of size n is recursively divided into two sub arrays of size n/2 until each of sub list size becomes 1. After complete divide process this method applies merging the two sub arrays recursively to obtain the single sorted list.
- The process of merging is combining the two sorted arrays into single sorted arrays. So,the process of merge sort involves following steps:
  - Divide the array into two subarrays recursively
  - Recursively sort the two subarrays
  - Combine two sorted subarrays into a single sorted array

To sort an array A[p . . q]:

- ***Divide***
  - Divide the n-element sequence to be sorted into two subsequences of n/2 elements each
- ***Conquer***
  - Sort the subsequences recursively using merge sort. When the size of the sequences is 1 there is nothing more to do

- ***Combine***
  - Merge the two sorted subsequences

Algorithm for Merge Sort

```
mergeSort(A, low, high){
        if(high>low){
          // Find the middle point to divide the array into two halves
              mid = (low+high)/2;
          //Call mergeSort for first half
              mergeSort(A, low, mid);
          //Call mergeSort for second half
              mergeSort(A, mid+1, high);
          //Merge the two halves sorted in above steps
              merge(A, low, mid, high);
        }
}
```

Algorithm for Merge

```
Merge(A, low, mid, high){
   i = low; j = mid+1;
   for(k = low; k<=high;k++){
        if(i > mid){
              B[k] = A[j];
              j = j+1;
        }
        else if(j > high){
              B[k] = A[i];
              i= i+1;
        }
        else if(A[i] <A[j] ){
```

```
            B[k] = A[i];
            i= i+1;
        }
        else{
            B[k] = A[j];
            j = j+1;
        }
    }
  for(k= low; k<=high;k++){
            A[k] = B[k];
        }
}
```

Example of merging procedure

Given two sorted subarrays of an array A are {1, 5, 10, 30, 50} and {3, 11, 60, 80}

	0	1	2	3	4	5	6	7	8
A [ ] =	**1**	**5**	**10**	**30**	**50**	**3**	**11**	**60**	**80**
	i = low				**mid**	**j**			**high**

	0	1	2	3	4	5	6	7	8
B [ ] =									
	k								

When k=0; A[i] < A[j] so, B[k] = A[i]; i++;

	0	1	2	3	4	5	6	7	8
B [ ] =	**1**								
	k								

0	1	2	3	4	5	6	7	8

A [ ] =	**1**	**5**	**10**	**30**	**50**	**3**	**11**	**60**	**80**
		i			**mid**	**j**			

Now k =1; A[i]> A[j] so, B[k] = A[j]; j++;

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
B [ ] =	**1**	**3**							
		k							

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
A [ ] =	**1**	**5**	**10**	**30**	**50**	**3**	**11**	**60**	**80**
		i			**mid**		**j**		

Now, k=2; A[i] < A[j] so, B[k] = A[i]; i++;

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
B [ ] =	**1**	**3**	**5**						
			k						

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
A [ ] =	**1**	**5**	**10**	**30**	**50**	**3**	**11**	**60**	**80**
			i		**mid**		**j**		

Now k=3; A[i] < A[j] so, B[k] = A[i]; i++;

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
B [ ] =	**1**	**3**	**5**	**10**					
				k					

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
A [ ] =	**1**	**5**	**10**	**30**	**50**	**3**	**11**	**60**	**80**
				i	**mid**		**j**		

When k =4; Here, A[i]> A[j] so, B[k] = A[j]; j++;

	0	1	2	3	4	5	6	7	8
B [ ] =	**1**	**3**	**5**	**10**	**11**				
					k				

	0	1	2	3	4	5	6	7	8
A [ ] =	**1**	**5**	**10**	**30**	**50**	**3**	**11**	**60**	**80**
				i	**mid**			**j**	

When k =5; Here, A[i] < A[j] so, B[k] = A[i]; i++;

	0	1	2	3	4	5	6	7	8
B [ ] =	**1**	**3**	**5**	**10**	**11**	**30**			
						k			

	0	1	2	3	4	5	6	7	8
A [ ] =	**1**	**5**	**10**	**30**	**50**	**3**	**11**	**60**	**80**
					i mid			**j**	

When k =6; Here, also A[i] < A[j] so, B[k] = A[i]; i++;

	0	1	2	3	4	5	6	7	8
B [ ] =	**1**	**3**	**5**	**10**	**11**	**30**	**50**		
							k		

	0	1	2	3	4	5	6	7	8
A [ ] =	**1**	**5**	**10**	**30**	**50**	**3**	**11**	**60**	**80**
					mid	**i**		**j**	

Now k =7; Here, i > mid so, B[k] = A[j]; j++;

	0	1	2	3	4	5	6	7	8
B [ ] =	**1**	**3**	**5**	**10**	**11**	**30**	**50**	**60**	
								k	

	0	1	2	3	4	5	6	7	8
A [ ] =	**1**	**5**	**10**	**30**	**50**	**3**	**11**	**60**	**80**

mid i j

When k =8; Here, i>mid so, B[k] = A[j]; j++;

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
B [ ] =	**1**	**3**	**5**	**10**	**11**	**30**	**50**	**60**	**80**

k

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
A [ ] =	**1**	**5**	**10**	**30**	**50**	**3**	**11**	**60**	**80**

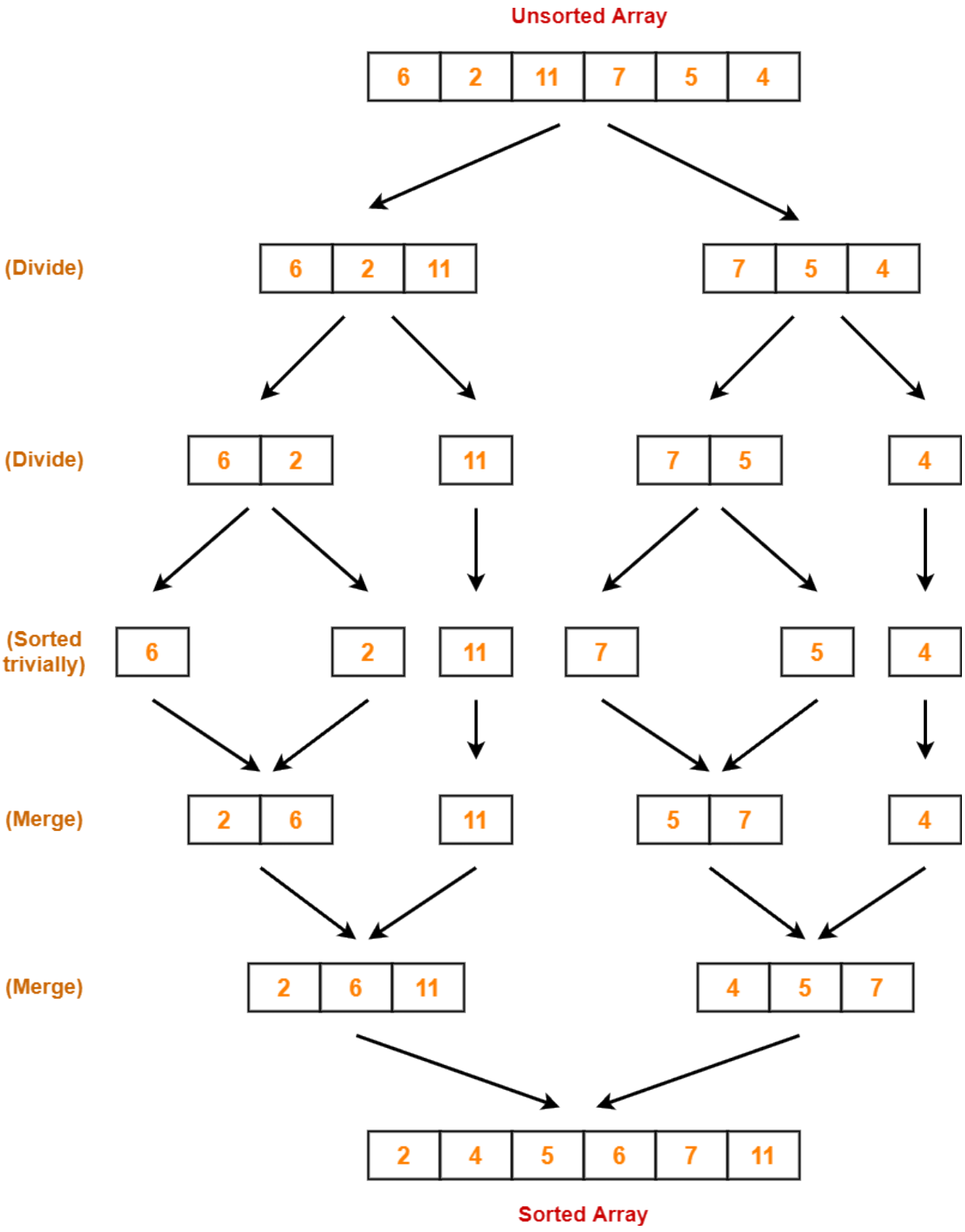
mid i j

Here k = high; so merging completed.

Now copying contents of temporary array B to the Original array A. Here the array is also sorted after combining sorted subarrays.

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
A [ ] =	**1**	**3**	**5**	**10**	**11**	**30**	**50**	**60**	**80**

Process of merge sort is illustrated in following figure


Unsorted Array
6 2 11 7 5 4
(Divide)
6 2 11
7 5 4
(Divide)
6 2
11
7 5
4
(Sorted trivially)
6
2
11
7
5
4
(Merge)
2 6
11
5 7
4
(Merge)
2 6 11
4 5 7
2 4 5 6 7 11
Sorted Array

Time Complexity:

- The time complexity of merge operation is linear since there is one scan of whole array for a complete merge of the list of size n.

  So, T(MERGE) = O(n)

- The time complexity of merge sort, T(n) = O(1)(for if and divide operation) +
  T(n/2) (for recursive call on left half) +
  T(n/2) (for recursive call on left half) +
  T(MERGE)

  $\therefore T(n) = 2T(n/2) + O(n)$

- Recurrence Relation for Merge sort:

  $T(n) = 1$ if n=1

  $T(n) = 2\ T(n/2) + O(n)$ if n>1

- Solving this recurrence we get

  Time Complexity = O(nlogn)

Space Complexity:

- It uses stack to store recursive function call records and one extra array for merging operations.

  Space Complexity= storage for recursive calls + size of extra array

  = O(logn) +O(n) (space for recursive call = depth of recursion tree = logn + 1)

  = O(n)

Heaps and Heap Sort

Heaps

- Heap is a special kind of binary tree where the nodes are arranged in specific way, depending on the type of heap.
- A particular kind of binary tree, called a heap, has the following two properties:
  1. All nodes are ordered in a specific way, depending on the type of heap. There are two types of heaps: **min heaps** and **max heaps.**

- In **max heap**, the value of each node is greater than or equal to the values stored in each of its children.
- In **min heap**, the value of each node is less than or equal to the values stored in each of its children.

2. The tree is perfectly balanced, and the leaves in the last level are all in the leftmost positions. (Complete binary tree)

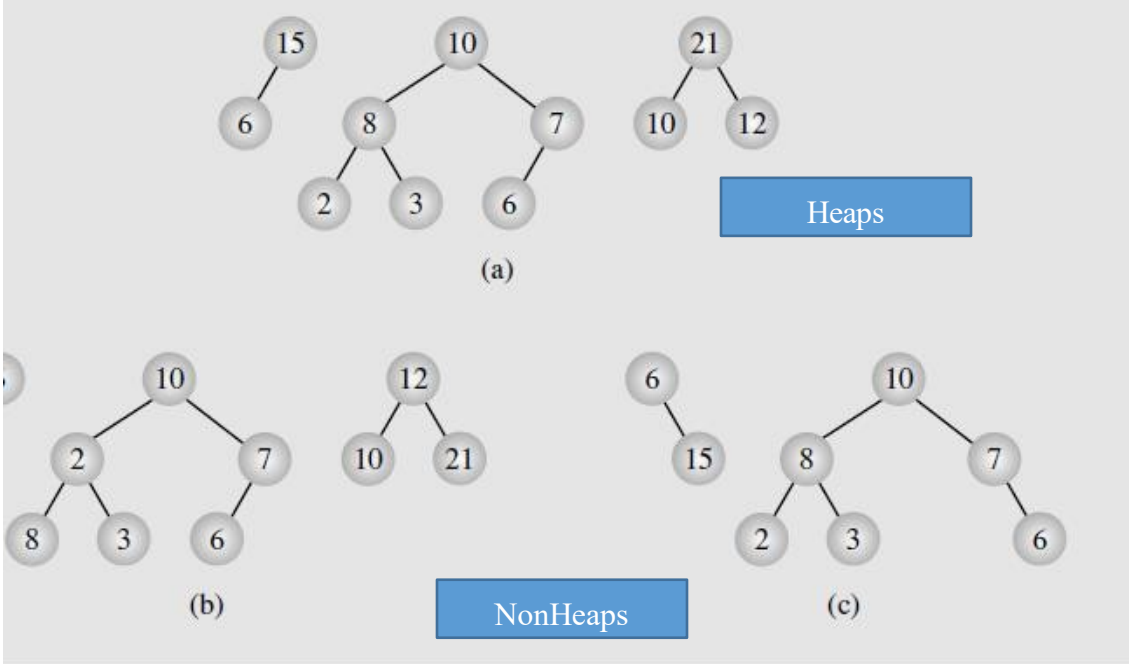


Figure: Examples of (a) heaps and (b–c) nonheaps

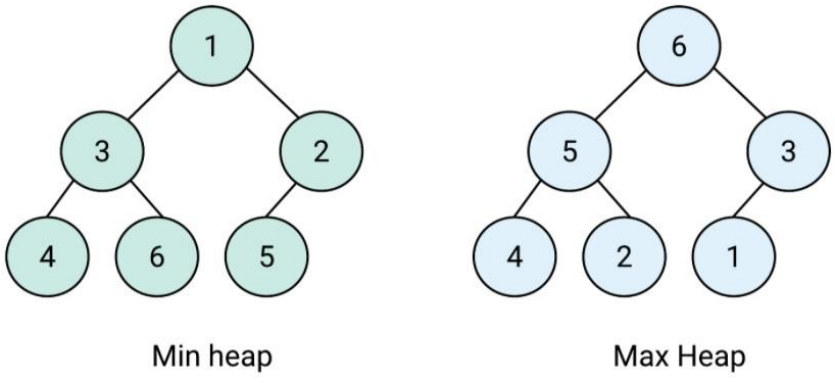
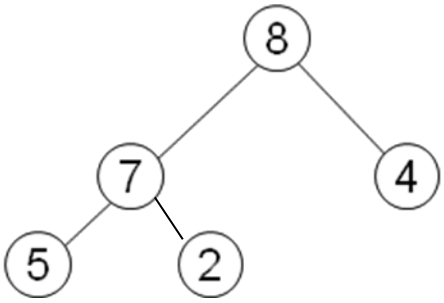


Figure: Examples of min heap and max heap

A **heap** is a nearly complete binary tree with the following two properties:

- **Structural property:** all levels are full, except possibly the last one, which is filled from left to right
- **Order (heap) property:** for any node x, Parent(x) ≥ x (for max heap)

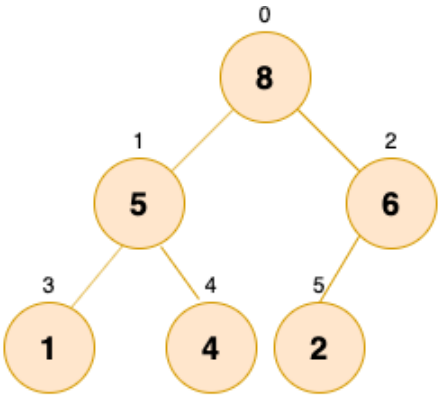


- Heaps are used to implement a priority queue and heap sorting.

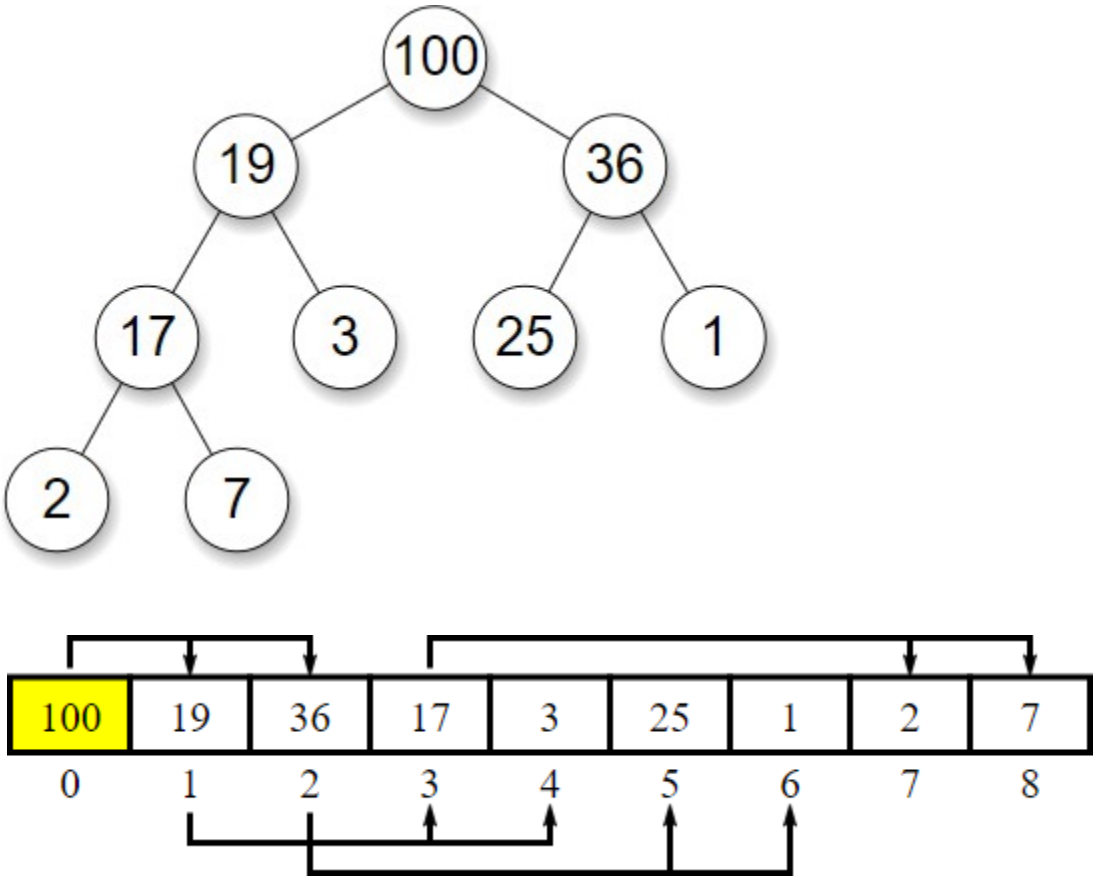
Array Representation of Heaps

- A heap can be stored as an array *A*.
  - Root of tree is A[0]
  - Left child of A[i] = A[2i+1]
  - Right child of A[i] = A[2i + 2]
  - Parent of A[i] = A[(i-1)/2]

- The traversal method use to achieve Array representation is Level Order(Breadth First Traversal i.e. level by level [Root==>Left==>Right])



8	5	6	1	4	2
0	1	2	3	4	5



Max-heaps (largest element at root), have the max-heap property:

– for all nodes i, excluding the root:

$$A[PARENT(i)] \geq A[i]$$

Min-heaps (smallest element at root), have the min-heap property:

– for all nodes i, excluding the root:

$$A[PARENT(i)] \leq A[i]$$

Adding/Deleting Nodes

New nodes are always inserted at the bottom level (left to right) and nodes are removed from the bottom level (right to left). After addition and deletion operation, the heap property must be maintained.

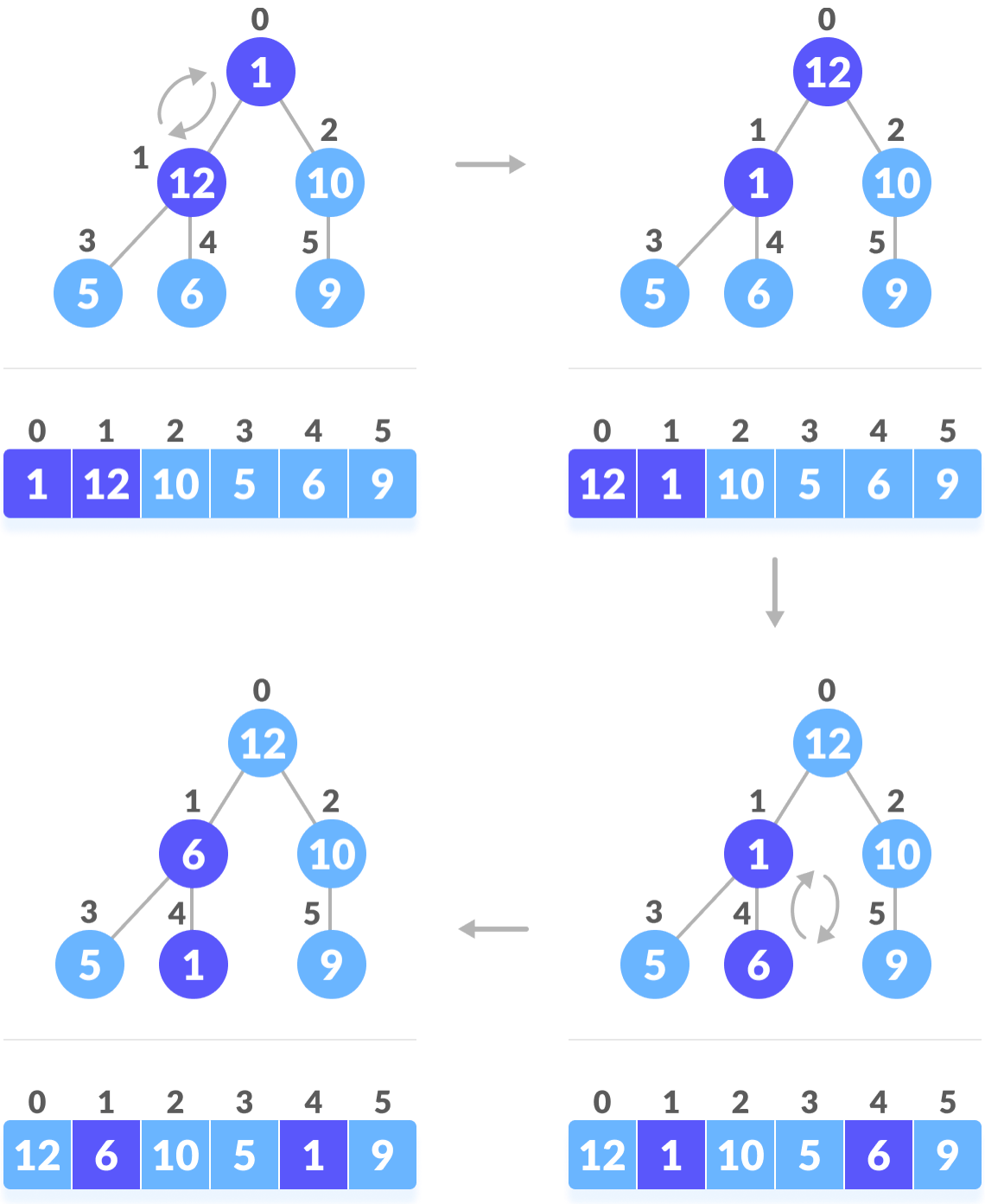
Operations on Heaps

- Maintain/Restore the max-heap property
  - MaxHeapify
- Create a max-heap from an unordered array
  - BuildMaxHeap
- Sort an array in place
  - HeapSort
- Priority queues

Maintaining the Heap Property

Suppose a node is smaller than a child and Left and Right subtrees of i are max-heaps. To eliminate the violation:

- Exchange with larger child
- Move down the tree
- Continue until node is not smaller than children


0
1
2
1
12
10
3
4
5
5
6
9
0 1 2 3 4 5
1 12 10 5 6 9
0
12
1
2
1
10
3
4
5
5
6
9
0 1 2 3 4 5
12 1 10 5 6 9
0
12
1
2
1
10
3
4
5
5
6
9
0 1 2 3 4 5
12 1 10 5 6 9
0
12
1
2
6
10
3
4
5
5
1
9
0 1 2 3 4 5
12 6 10 5 1 9

Algorithm:

```
MaxHeapify(A, i, n) // i is a node index
{
     left = 2*i+1; // left child index
     right = 2*i+2;// right child index
     largest=i;
     if (left ≤ n and A[left] > A[largest]);
          largest = left;
     if (right ≤ n and A[right] > A[largest])
          largest = right;
     if (largest!= i){
          swap(A[i] , A[largest])
          MaxHeapify(A, largest, n)
     }
}
```

Analysis:

In the worst case MaxHeapify is called recursively h times, where h is height of the heap and since each call to the heapify takes constant time.

Time complexity = O(h) = O(logn)

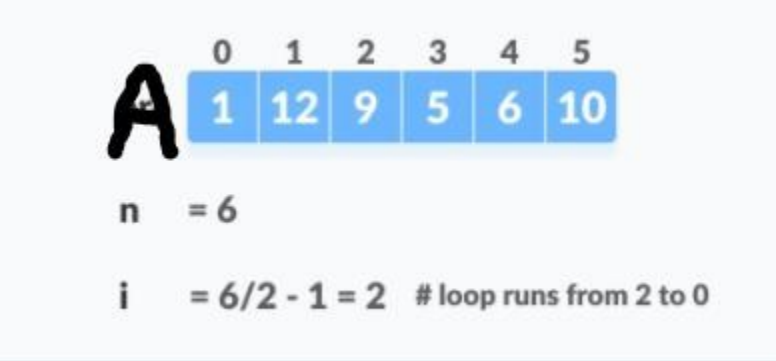
Building a Heap

Convert an array A[0 … (n-1)] into a max-heap (n = length[A]). The elements in the subarray A[(⌊n/2⌋) … (n-1)] are leaves. Apply MaxHeapify on elements at index 0 to (⌊n/2⌋-1) .

Algorithm:

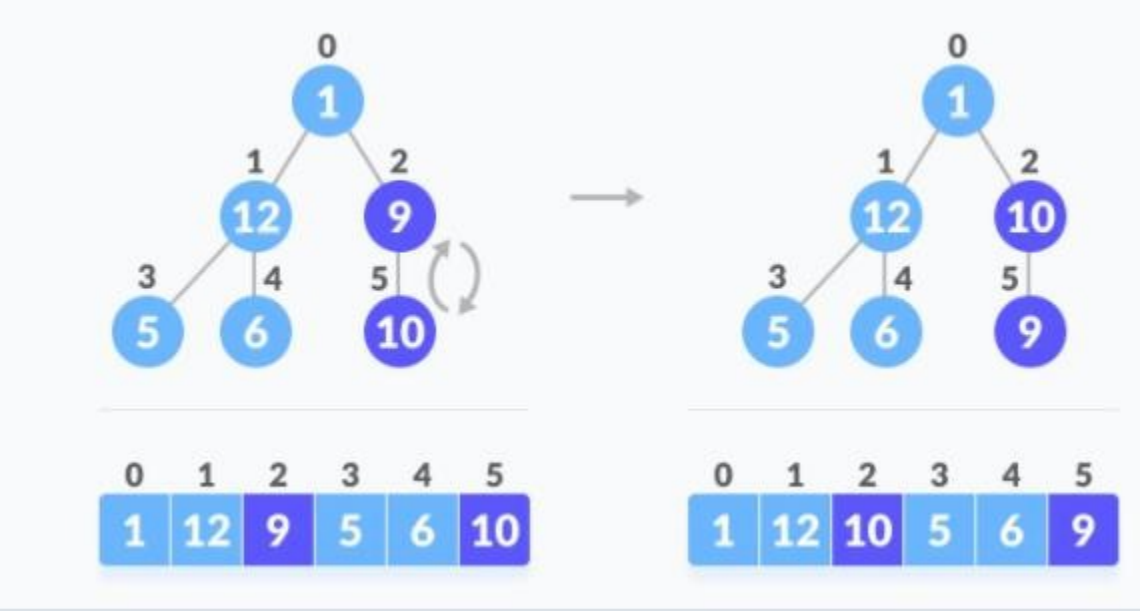
```
BuildMaxHeap(A){
     n = length(A)
     for(i = n/2-1;i>=0;i--){
          MaxHeapify(A,i,n);
     }
}
```

Example:



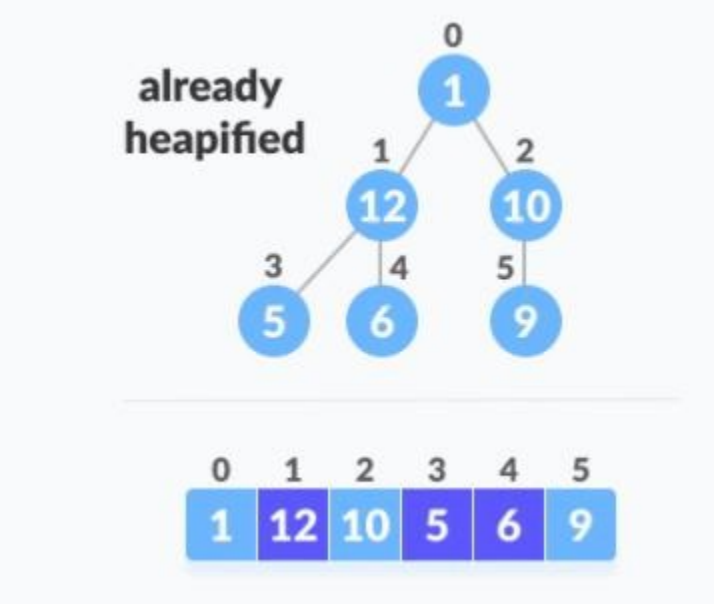
When i = 2

MaxHeapify(A,2,6)



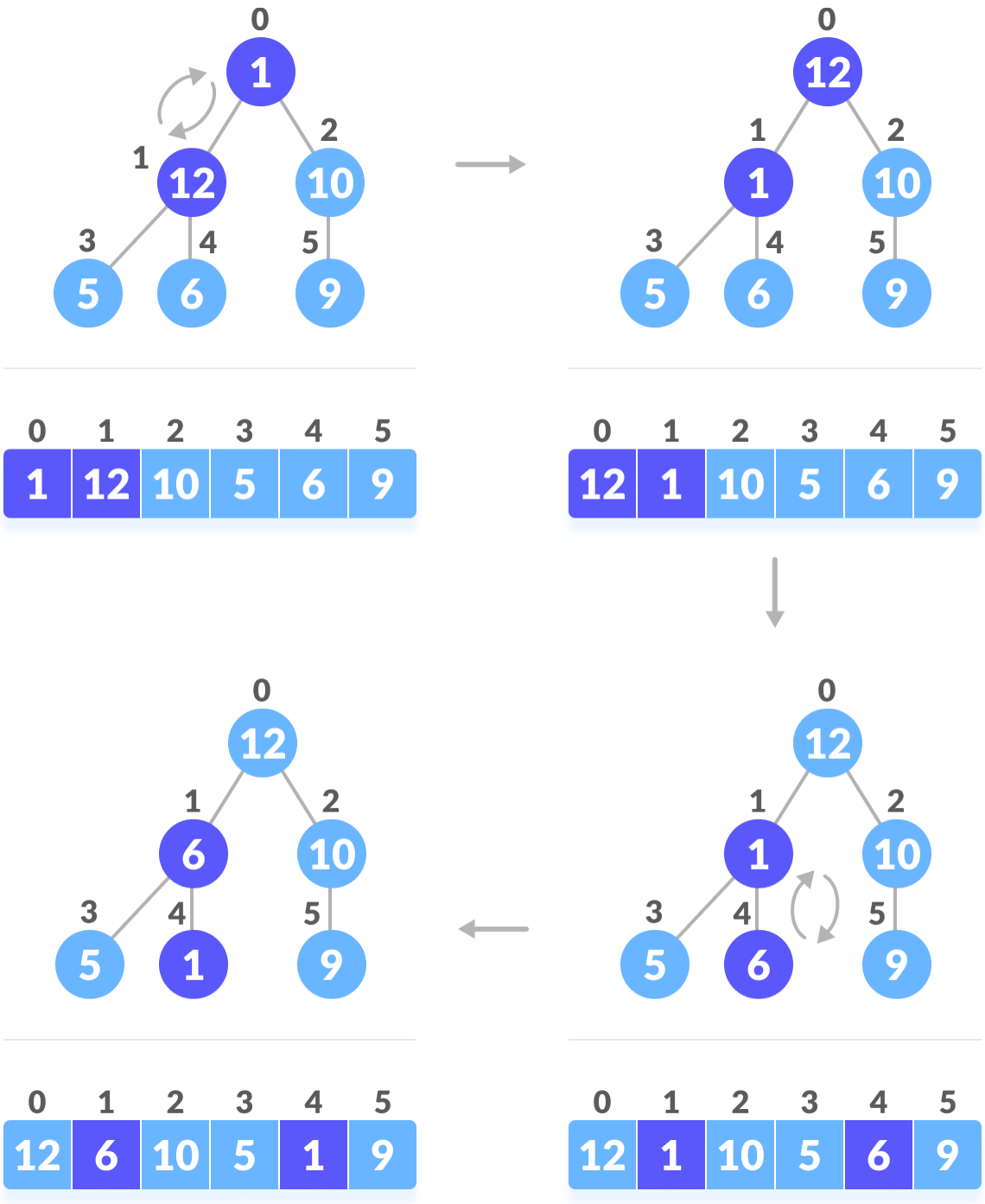
When i = 1

MaxHeapify(A,1,6)



When i = 0

MaxHeapify(A,0,6)


0
1
2
1
12
10
3
4
5
5
6
9
0 1 2 3 4 5
1 12 10 5 6 9
0
12
1
2
1
10
3
4
5
5
6
9
0 1 2 3 4 5
12 1 10 5 6 9
0
12
1
2
1
10
3
4
5
5
6
9
0 1 2 3 4 5
12 1 10 5 6 9
0
12
1
2
6
10
3
4
5
5
1
9
0 1 2 3 4 5
12 6 10 5 1 9

Time Complexity:

- We can compute a simple upper bound on the running time of BuildMaxHeap as follows.
  - — Each call to MaxHeapify costs O(lg n) time, and BuildMaxHeap makes O(n) such calls.
  - — Thus, the running time is O(n lg n).
  - — This upper bound, though correct, is **not asymptotically tight.**.

  (As we know that heapify is called for all internal nodes, and heapify takes O(logn) time, but this is not exactly the case. In reality, the O(logn) time is taken by root only, as root is at the height logn, time varies for the internal nodes with different height. For example, for the internal nodes at the second last level, heapify will take O(h) time for all the nodes at that level where h=1, for the internal nodes at the third last level, heapify will take O(h) time for all the nodes at that level where h=2, therefore depending height time to heapify is changing. So considering this, ***we can derive a tighter upper bound than O(nLogn).)***
- We can derive a tighter bound by observing that the *time for MaxHeapify to run at a node varies with the height of the node in the tree, and the heights of most nodes are small*.
- Our tighter analysis relies on following properties:
  - — **An n-element heap has height $\lfloor \log_2 n \rfloor$ and**
  - — **At height h there are at most $\lceil \frac{n}{2^{h+1}} \rceil$ nodes**
- The time required by MaxHeapify called on a node of height h is O(h)

  So, The total cost,

$$T(n) = \sum_{h=0}^{\lfloor \lg n \rfloor} \lceil \frac{n}{2^{h+1}} \rceil O(h)$$

$$= O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right)$$

  It is clear that,

$$n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \leq n \sum_{h=0}^{\infty} \frac{h}{2^h}$$

  So, We can bound the running time of the algorithm as

$$T(n) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right)$$

We know,

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x} \quad for\ x < 1$$

Differentiating on both sides we get,

$$\sum_{k=0}^{\infty} k\, x^{k-1} = \frac{1}{(1-x)^2}$$

$$\sum_{k=0}^{\infty} k\, x^k = \frac{x}{(1-x)^2}$$

When $x = 1/2$,

$$\sum_{k=0}^{\infty} k \left(\frac{1}{2}\right)^k = \frac{\frac{1}{2}}{\left(1-\frac{1}{2}\right)^2}$$

$$\sum_{k=0}^{\infty} \frac{k}{2^k} = \frac{\frac{1}{2}}{\left(1-\frac{1}{2}\right)^2}$$

$$= 2$$

$$\therefore \quad \sum_{h=0}^{\infty} \frac{h}{2^h} = 2$$

Hence,

$$T(n) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right)$$

$$= O(n * 2)$$

$$= O(n)$$

Heapsort

To have the array sorted in ascending order, heap sort puts the largest element at the end of the array, then the second largest in front of it, and so on. Heap sort starts from the end of the array by finding the largest elements.

Process to follow in heap sort:

- Build a max-heap from the array
- Swap the root (the maximum element) with the last element in the array
- Discard this last node by decreasing the heap size
- Call MaxHeapify on the new root
- Repeat this process until all the items of the list are sorted (only one node remains)

Note: For descending order, Min heaps should be used

Pseudocode

```
HeapSort(A){
    BuildMaxHeap(A); //convert the array into max heap n = length(A);
    for(i = n-1 ; i >= 1; i--){
        swap(A[0],A[n-1]);//swap root and last element n = n-1; //
        decrease the size of heap MaxHeapify(A,0,n);
    }
}
```

Time complexity of HeapSort

To convert the array into max heap, BuildMaxHeap() is used which takes O(n) steps. MaxHeapify() takes O(lgn) steps for one call and it is called O(n) times.

So Time complexity, T(n) = O(n) + O(lgn)*O(n)

= O(n) + O(nlgn)

= O(nlgn)

Example: Sort the following array A using HeapSort

A =

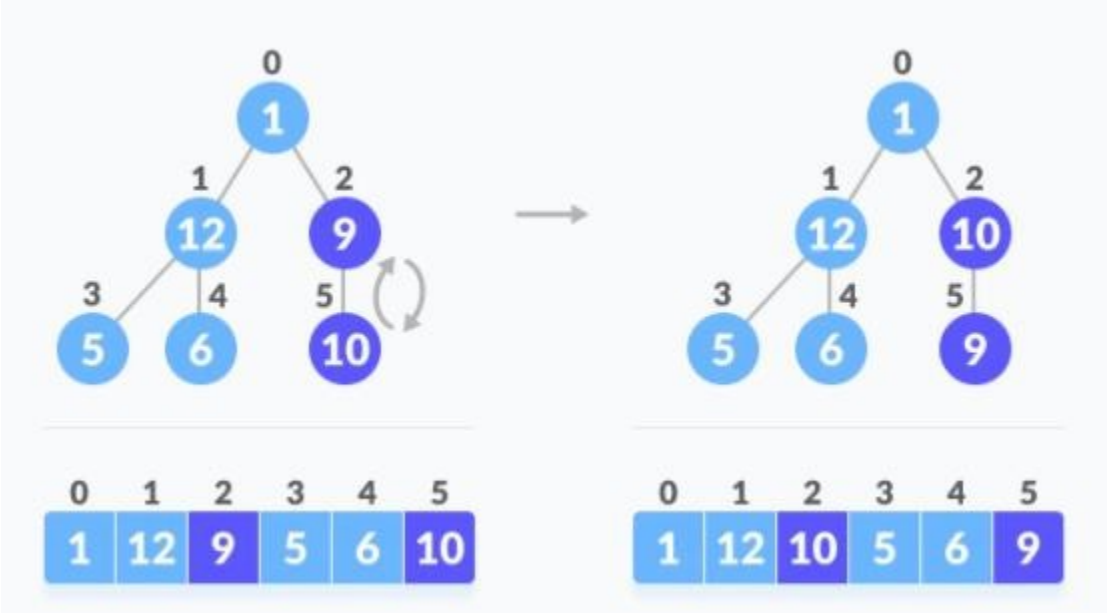
0	1	2	3	4	5
1	12	9	5	6	10

Creating max heap from the given array by calling BuildMaxHeap(A)

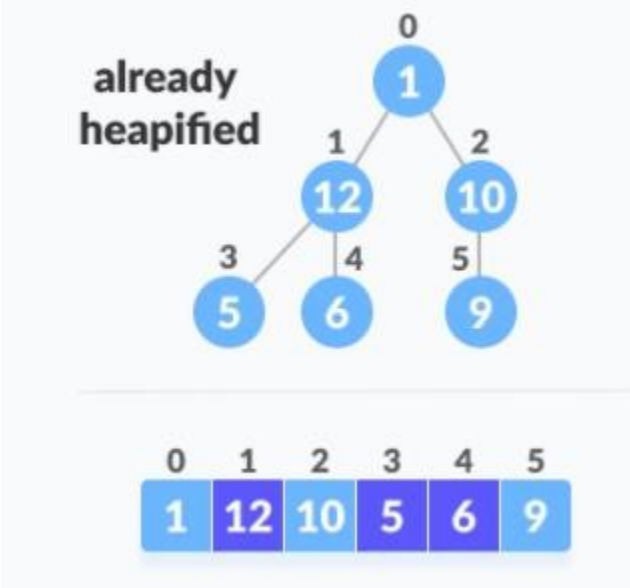
Here n = lenngth(A) = 6

Calling MaxHeapify(A,i,n) for i = (n/2-1) i.e (6/2 -1) = 2 to 0

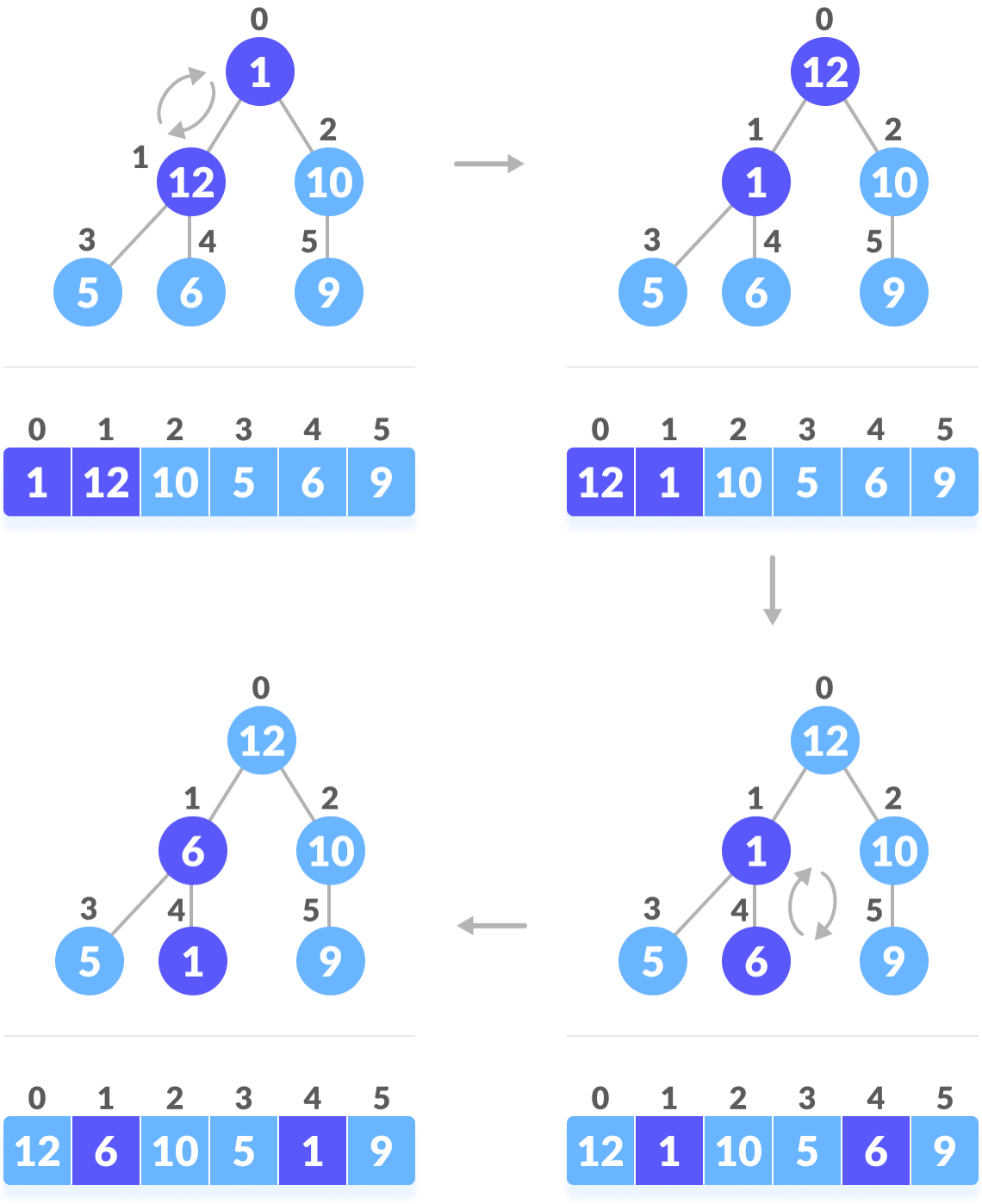
✍ **MaxHeapify(A,2,6) [here i = 2]**



✍ **MaxHeapify(A,1,6) [here i = 1]**



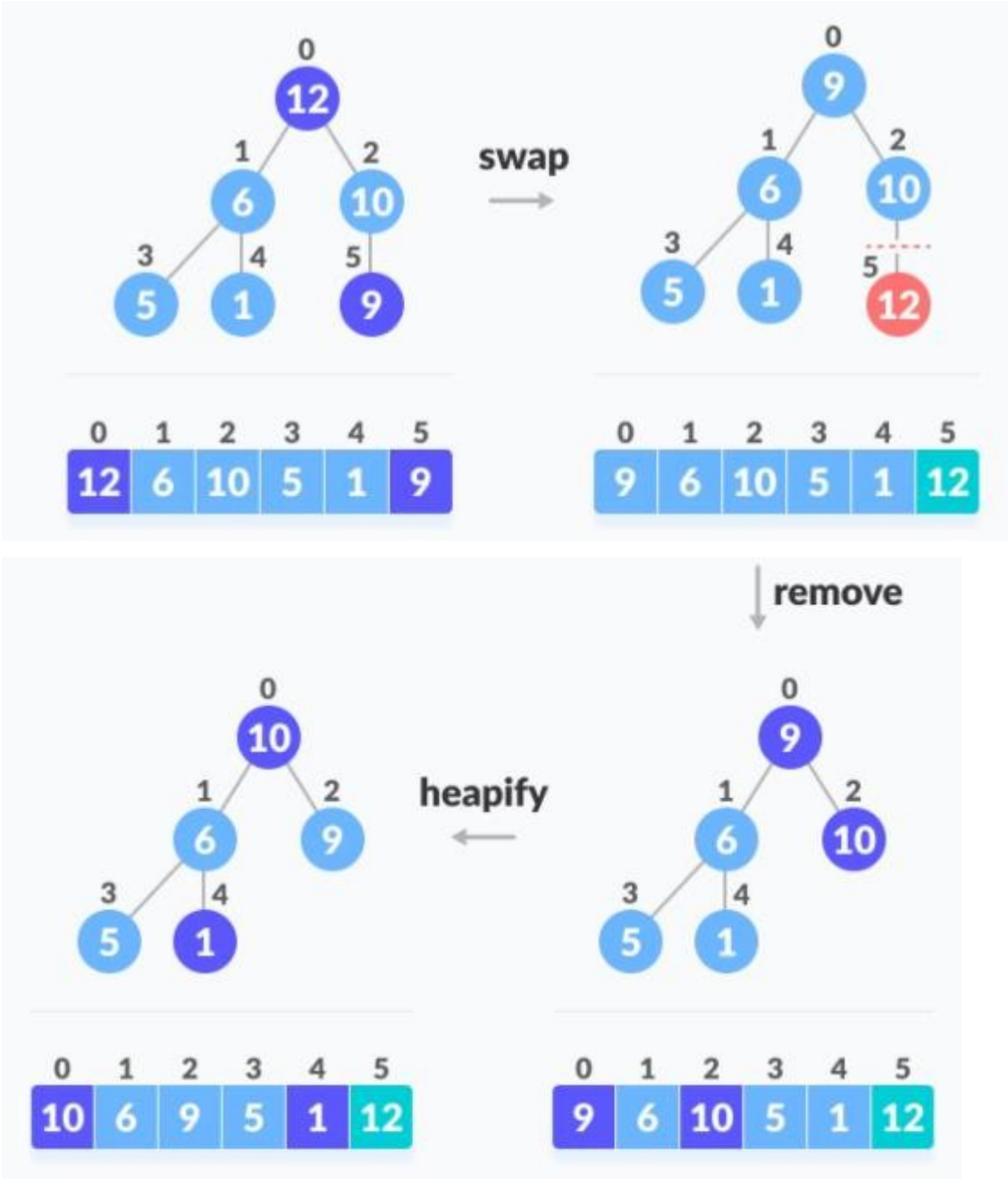
MaxHeapify(A,0,6) [here i = 0]



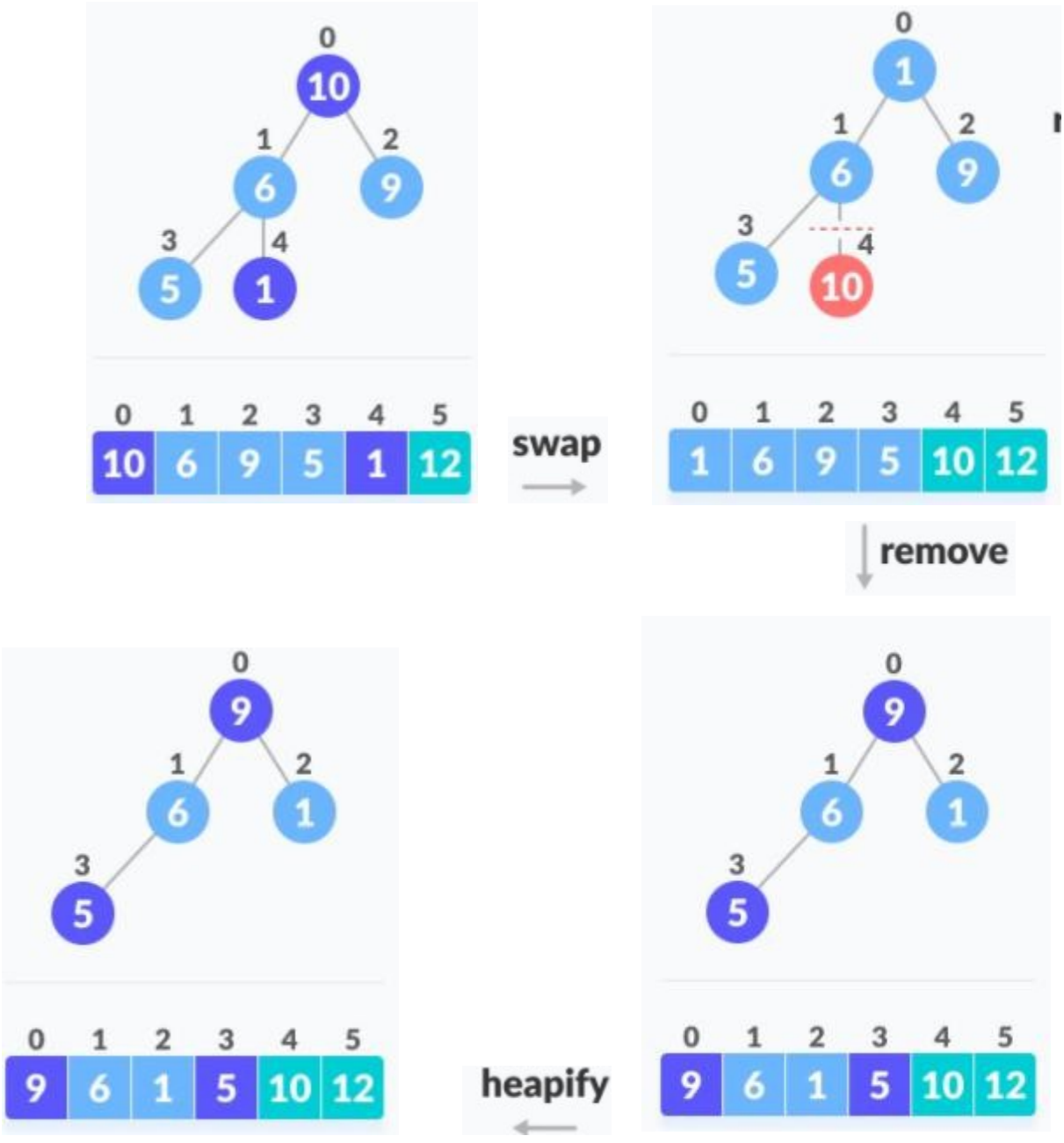
Now, max heap has been created.

For i = n-1 to 1 (i.e.i= 6-1=5 to 1) we have to swap root element with last element (i.e. element at n-1 position) and discard (remove) the last element by decreasing heap size (i.e by setting n = n-1)

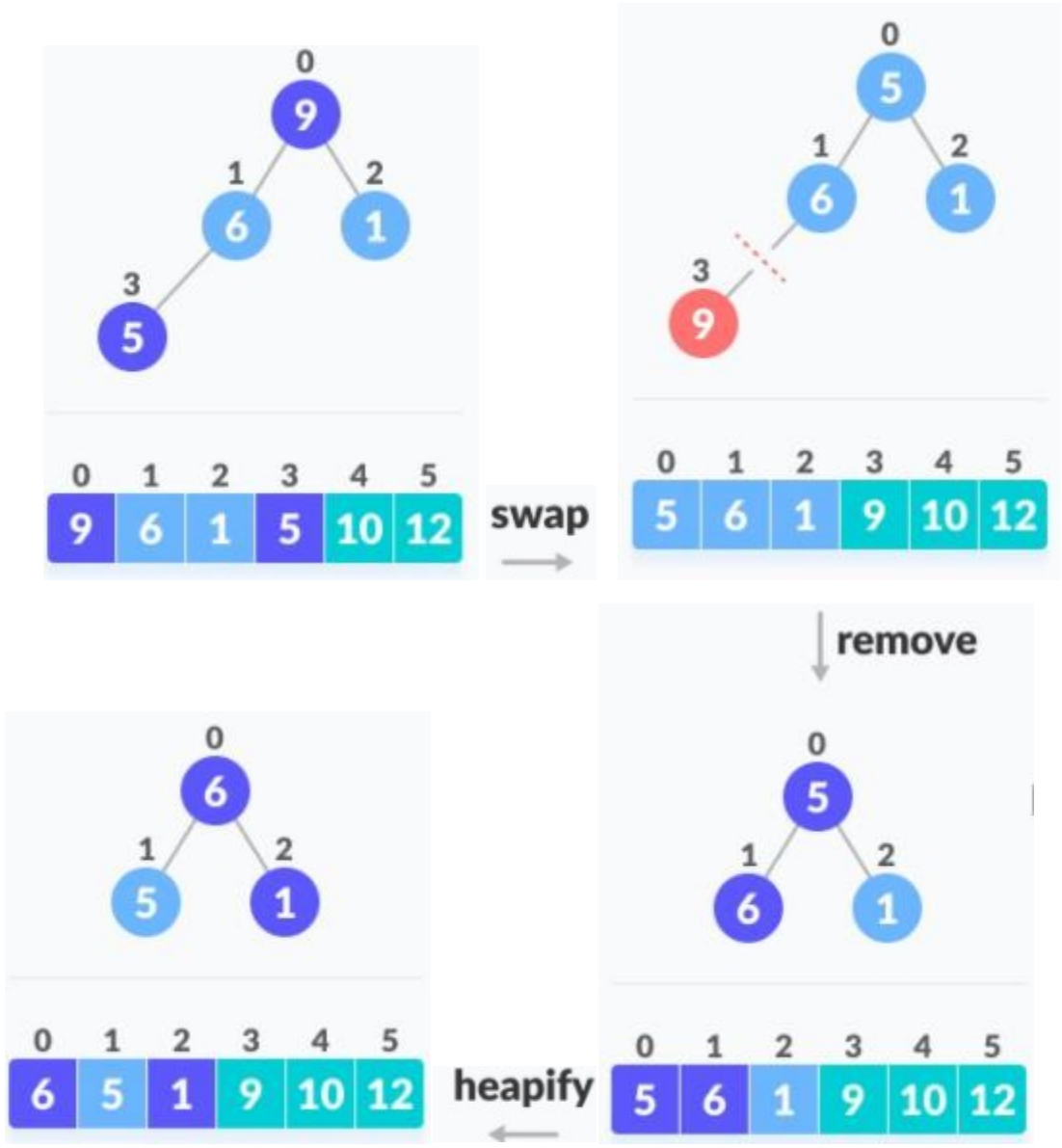
- **[for i = 5]** Swap (A[0], A[5]) and discard A[5] (i.e. n = 5-1 =4) and heapify by calling MaxHeapify(A,0, 4)



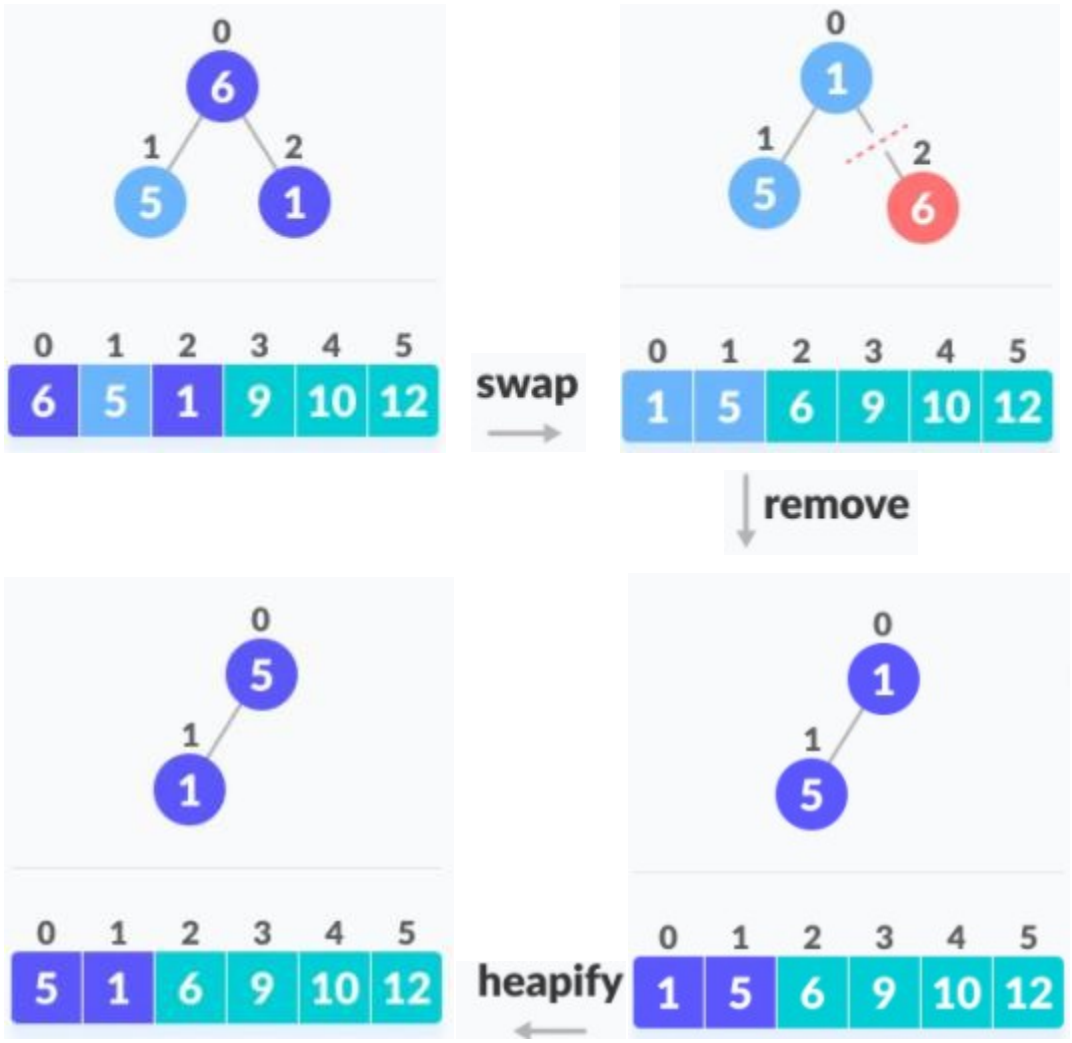
- **[for i = 4]** Again Swap (A[0], A[4]) and discard A[4] (i.e. n = 4-1 =3) and heapify by calling MaxHeapify(A,0, 3)



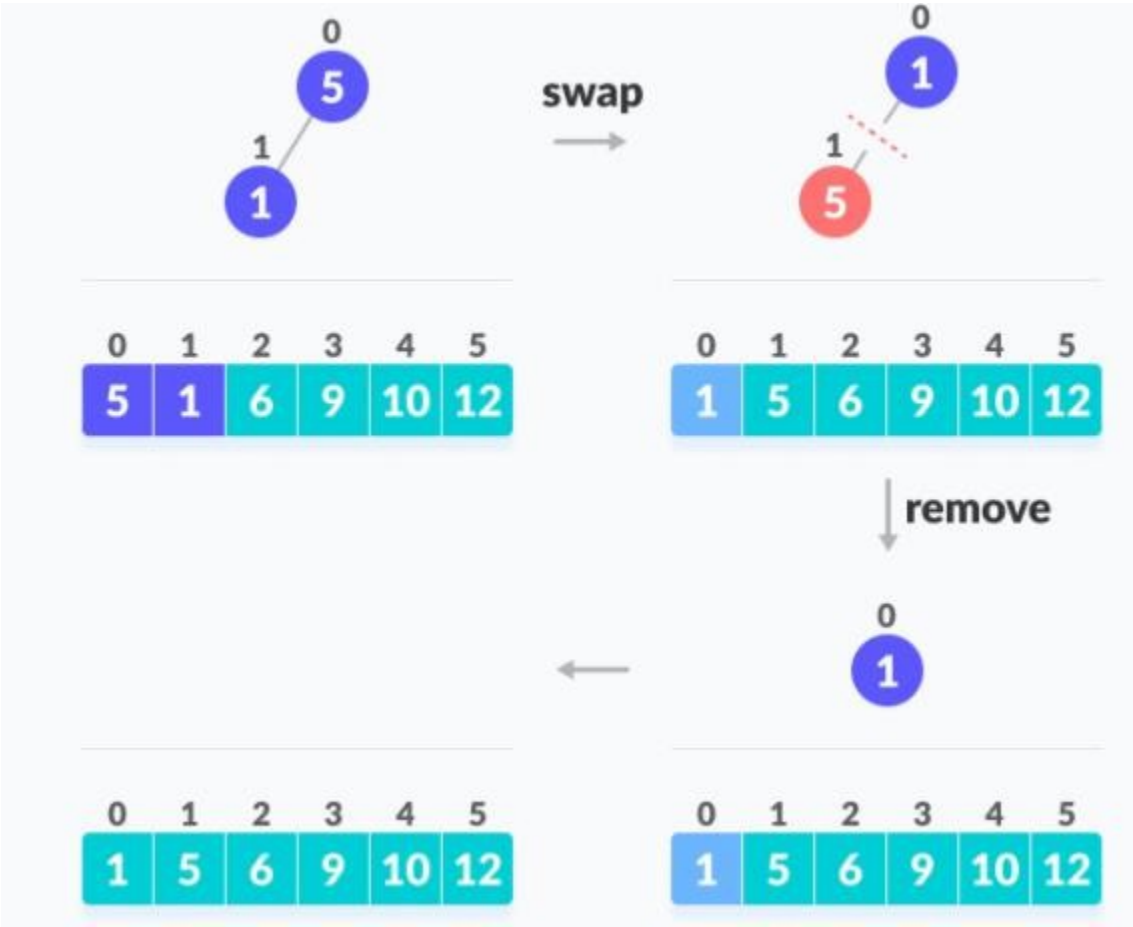
- **[for i = 3]** Again Swap (A[0], A[3]) and discard A[3] (i.e. n = 3-1 =2) and heapify by calling MaxHeapify(A,0, 2)



- **[for i = 2]** Again Swap (A[0], A[2]) and discard A[2] (i.e. n = 2-1 =1) and heapify by calling MaxHeapify(A,0, 1)



☜ **[for i = 1]** Again Swap (A[0], A[1]) and discard A[1] (i.e. n = 1-1 =0) and since n=1, stop heapifying.



The array is now sorted in ascending order.

NOTE: To sort in descending order we have to use min heap.

Selection Problem and Order Statistics

Selection Problem

Given a set A of n distinct elements and an integer i such that $1 \leq i \leq n$. Find an element x of set A which is exactly greater than (or smaller than) i – 1 elements of the set. i.e. finding the i^{th} largest(or smallest) element of the set A.

- We can solve the problem using following approach *:*
  - Sort the elements using any efficient sorting algorithm
  - Index the i^{th} element in output of sorting
- This solution takes O(n lg n) time since the complexity of any efficient algorithm is O(n lg n) and selecting the i^{th} element in sorted list requires O(1) time.

Order Statistics

- Order Statistics refers to statistical methods that depend on ordering of the data.
- .Selecting the i^{th} smallest (or largest) element from a given list of size n is called i^{th} order statistic.
- i^{th} order statistic of a set of elements gives i^{th} largest(smallest) element.
- In general let's think of i^{th} order statistic gives i^{th} smallest.
- For example, the ***minimum*** of a set of elements is the first order statistic (i =1), and the ***maximum*** is the nth (last) order statistic (i = n).
- A ***median***, informally, is the "halfway point" of the set.
  - When n is odd, the median is unique, occurring at i = (n+1)/2.
  - When n is even, there are two medians, occurring at n/2 and n/2 + 1
- A **median is given by i^{th} order statistic where i = (n+1)/2 for odd n and i = n/2 and n/2 + 1 for even n.**
- For the median finding problem the value of i = (n+1)/2 for the list of odd size and i = n/2 and n/2 + 1 for the list of even size. This is known as **median order statistics**

NOTE: There are other methods to solve the selection problem efficiently than O(n lg n)

Brute Force approach for solving selection problem (Nonlinear general selection algorithm)

We can construct a simple, but inefficient general algorithm for finding the *k*th smallest or *k*th largest item in a list, requiring O(*kn*) time, which is effective when *k* is small. To accomplish this, we simply find the most extreme value and move it to the beginning until we reach our desired index.

```
Select(A, k)
{
        for( i=0;i<k;i++)
        {
            mi= i;     //minindex
            mv = A[i];         //minvalue
           for(j=i+1;j<n;j++)
           {
              if( A[j] < mv)
              {
                    mi = j;
                    mv = A[j];
              }
             swap(A[i],A[mi]);
         }
         return A[k-1];
}
```

Example:

A = {44, 20, 15, 77, 5, 36} and we have to find 3rd smallest element in array A.

Here

0	1	2	3	4	5
44	20	15	77	5	36

n =6 (number of elements)

k = 3 (3rd smallest element is to be found)

Value of i ranges from 0 to 2(since i<k)

When i =0

mi= i= 0

mv = A[i] = A[0] = 44

j= i+1 = 0+1 = 1 to 5 (since j<n)

j	mi	mv	A[j]<mv	Array
1	0	44	A[1]<44 is true mi =1 mv = A[1]=20	0 1 2 3 4 5 [ ][ ] 15 77 5 36
2	1	20	A[2]<20 is true mi =2 mv = A[2]=15	0 1 2 3 4 5 44 [ ][ ] 77 5 36
3	2	15	A[3]<15 is false	0 1 2 3 4 5 44 20 [ ][ ] 5 36
4	2	15	A[4]<15 is true mi =4 mv = A[4]=5	0 1 2 3 4 5 44 20 [ ] 77 [ ] 36
5	4	5	A[5]< 5 is false	0 1 2 3 4 5 44 20 15 77 5 36

Now swap (A[i],A[mi]) i.e swap (A[0],A[4])

0	1	2	3	4	5
44	20	15	77	5	36

0	1	2	3	4	5
5	20	15	77	44	36

When i =1

mi= i= 1

mv = A[i] = A[1]=20

j= i+1 = 1+1 = 2 to 4 (since j<n)

j	mi	mv	A[j]<mv	Array
2	1	20	A[2]< 20 is true mi =2 mv = A[2]=15	0 1 2 3 4 5 5 77 44 36
3	2	15	A[3]< 15 is false	0 1 2 3 4 5 5 20 44 36
4	2	15	A[4]< 15 is false	0 1 2 3 4 5 5 20 77 36
5	2	15	A[5]< 15 is true	0 1 2 3 4 5 5 20 77 44

Now swap (A[i],A[mi]) i.e swap (A[1],A[2])

0	1	2	3	4	5
5	20	15	77	44	36

0	1	2	3	4	5
5	15	20	77	44	36

When i =2

mi= i= 2

mv = A[i] = A[2]=15

j= i+1 = 2+1 = 3 to 5 (since j<n)

j	mi	mv	A[j]<mv	Array
3	2	20	A[3]< 15 is false	0 1 2 3 4 5 5 15 44 36
4	2	20	A[4]< 15 is false	0 1 2 3 4 5 5 15 77 36
5	2	20	A[5]< 15 is false	0 1 2 3 4 5 5 15 77 44

Now swap (A[i],A[mi]) i.e swap (A[2],A[2])

0	1	2	3	4	5
5	15	20	77	44	36

0	1	2	3	4	5
5	15	20	77	44	36

Now the 3rd smallest element = A[k-1] = A[3-1] = A[2] = 20 #

NOTE: i^{th} largest = (n-i+1) smallest

Analysis:

When i=0, inner loop executes n-1 times

When i=1, inner loop executes n-2 times

When i=2, inner loop executes n-3 times

……...

When i=k-1 inner loop executes n-(k+1) times

Thus, Time Complexity = (n-1) + (n-2) +.....(n-k-1)

= O(kn) ≈ $O(n^2)$

Selection in expected linear time

This problem is solved by using the "divide and conquer" method. The main idea for this problem solving is to partition the element set as in Quick Sort where partition is randomized one.

The following pseudocode for RandmizedSelect returns the i^{th} smallest element of the array A[p,...,r].

```
RandmizedSelect (A,p,r,i)
{
        if(p = =r )
                return A[p];
        q = RandmizedPartition(A,p,r);
        k = q – p + 1;
        if(i = = k) // the pivot value is the answer
                return A[q]
        else if(i < k)
                return RandmizedSelect (A, p, q - 1, i);
        else
                return RandmizedSelect (A, q + 1, r, i - k);
}
```

Example: Find the 2^{nd} smallest element of the list A = {70, 10, 20, 5, 40}

Here , i = 2

RandmizedSelect (A, 0, 4, 2);

p≠r

q = RandomizedPartition(A,0,4)

Let random partition index (random pivot) = 2 then the list is partitioned from index 2.

Exchange pivot with first element

0	1	2	3	4
70	10	20	5	40

After exchange

0	1	2	3	4
20	10	70	5	40
L				**R**

Performing Partitioning

Pivot = 20

0	1	2	3	4
20	10	70	5	40
		L	**R**	

L<R swap A[L] and A[R]

0	1	2	3	4
20	10	5	70	40
		L	**R**	

0	1	2	3	4
20	10	5	70	40
		R	**L**	

L>R swap A[R] and pivot

0	1	2	3	4
5	10	20	70	40

Here , partitioning index q = 2

Now k = q-p+1

2+0+1 = 3

Since i < k, repeat the above steps on left half i.e RandmizedSelect (A, p, q - 1, i);

RandmizedSelect (A, 0, 1 2);

p≠r

q = RandomizedPartition(A,0,1)

0	1	2	3	4
5	10	20	70	40

Let random pivot = 1;

Exchanging 5 and 10

0	1	2	3	4
10	5	20	70	40
L	**R**			

Pivot = 10

0	1	2	3	4
10	5	20	70	40
	L			
	R			

L==R, swap A[R] and pivot

0	1	2	3	4
5	10	20	70	40

Here , partitioning index q = 1

Now k = q-p+1

= 1- 0+1 = 2

Since i = = k ,

The 2nd smallest element = A[q] = A[1] = 10 ###

Analysis:

Since our algorithm is randomized algorithm no particular input is responsible for worst case however the worst case running time of this algorithm is $O(n^2)$. This happens if every time unfortunately the pivot chosen is always the largest one (if we are finding minimum element). Assume that the probability of selecting pivot is equal to all the elements i.e 1/n then we have the recurrence relation,

$$T(n) = 1/n\left(\sum_{j=1}^{n-1} T(\max(j,n-j))\right) + O(n)$$

Where, max(j,n-j) = j, if j>= ceil(n / 2)

and max(j,n-j) = n-j, otherwise.

Observe that every T(j) or T(n – j) will repeat twice for both odd and even value of n (one may not be repeated) one time form 1 to ceil(n / 2) and second time for ceil(n / 2) to n-1, so we can write,

$$T(n) = 2/n\left(\sum_{j=n/2}^{n-1} T(j)\right) + O(n)$$

Using substitution method,

Guess T(n) = O(n)

To show T(n) <= cn

Basic Step: T(1) <= c*1

1<=c which is true for all positive value of c

Inductive step: Let's assume that is true for all j<n

then T(j) <= cj

Substituting on the relation

$$T(n) = 2/n \sum_{j=n/2}^{n-1} cj + O(n)$$

$$T(n) = 2/n \{\sum_{j=1}^{n-1} cj - \sum_{j=1}^{n/2-1} cj \}+ O(n)$$

$$T(n) = 2/n \{ (n(n-1))/2 –((n/2-1)n/2)/2\} + O(n)$$

$$T(n) \leq c(n-1) – c(n/2-1)/2 + O(n)$$

$$T(n) \leq cn – c – cn/4 + c/2 +O(n)$$

$$= cn – cn/4 –c/2 + O(n)$$

$$\leq cn \{\text{choose the value of c such that } (-cn/4-c/2 +O(n) \leq 0 \}$$

$\Longrightarrow$ T(n) = O(n)

Selection in worst case linear time

- In *RandomizedSelect* , it is not guaranteed that the partition is good. So, for worst case, it runs in $O(n^2)$.
- If some algorithm guarantees the good partition recursively then the selection is always expected in linear time.
- So, for the worst case, to guarantee the running time to be linear, an algorithm can be used as computing the median of medians of small groups and taking the median of medians as partitioning pivot.
- *Main Idea: Divide the n elements into groups of 5. Find the median of each 5-element group. Recursively Select the median x of the $\lfloor n/5 \rfloor$ group medians to be the pivot.*

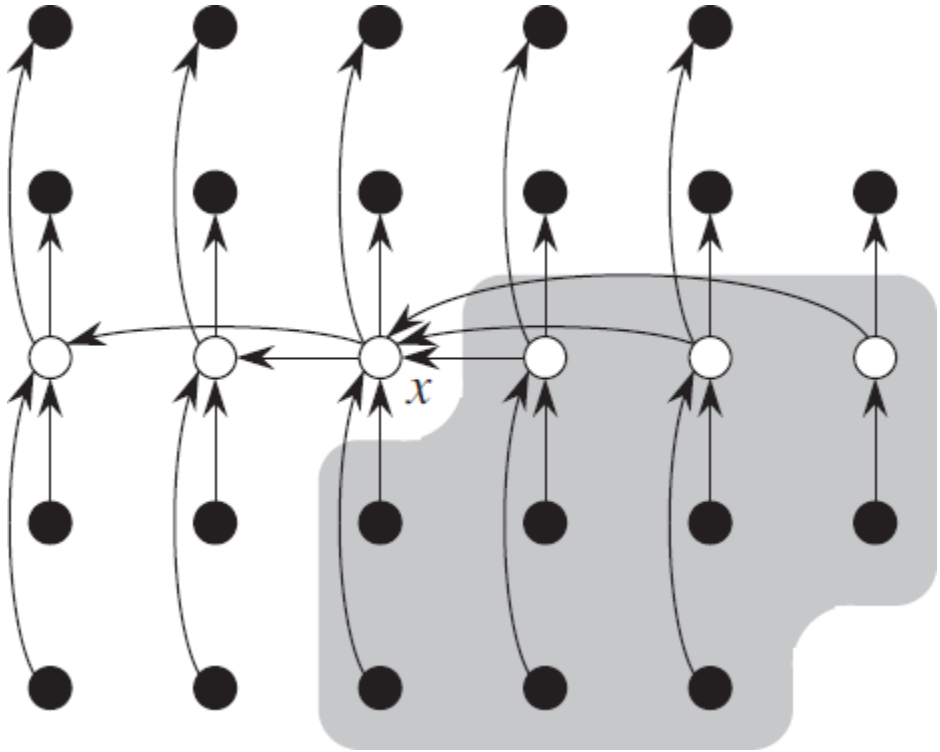
- The algorithm can be outlined as

```
Select (A,p,r,i):
```

1. Divide the n elements of the input array into $\lceil \frac{n}{5} \rceil$ groups of 5 elements each and at most one group made up of the remaining $n \bmod 5$ elements.
2. Find the median of each of the $\lceil \frac{n}{5} \rceil$ groups by first insertion-sorting the elements of each group (of which there are at most 5) and then picking the median from the sorted list of group elements.
3. Use Select () recursively to find the median x of the $\lceil \frac{n}{5} \rceil$ medians found in step 2. (If there are an even number of medians, then by our convention, x is the lower median.) Take x as pivot for partitioning
4. Let k = rank (x) [i.e. position of x] . Suppose k is one more than the number of elements on the low side (left) of the partition , so that x is the k^{th} smallest element and there are n-k elements on the high side (right) of the partition

5. If k==i then return x. Otherwise, use Select() recursively to find the i^{th} smallest element on the low side if i < k, or the $(i - k)^{th}$ smallest element on the high side if i > k.
   - if k = = i then return x
   - esle if i < k then use Select() recursively to find i^{th} smallest element in first partition (left side)
   - else (i > k) use Select() recursively to find $(i-k)^{th}$ smallest element in last partition (right side)

This approach can be visualized as in figure below:



- The n elements are represented by small circles, and each group of 5 elements occupies a column.
- The medians of the groups are whitened, and the median-of-medians x is labeled. (When finding the median of an even number of elements, we use the lower median.)

- Arrows go from larger elements to smaller, from which we can see that 3 out of every full group of 5 elements to the right of x are greater than x, and 3 out of every group of 5 elements to the left of x are less than x.
- The elements known to be greater than x appear on a shaded background.

To analyze the running time of Select (), we first determine a lower bound on the number of elements that are greater than the partitioning element x.

- At least half of the medians found in step 2 are greater than or equal to the median-of-medians x.
- Thus, at least half of the $\lceil \frac{n}{5} \rceil$ groups contribute at least 3 elements that are greater than x, except for the one group that has fewer than 5 elements if 5 does not divide n exactly, and the one group containing x itself. Discounting these two groups, it follows that the number of elements greater than x is at least

$$3\left(\left\lceil \frac{1}{2}\left\lceil \frac{n}{5}\right\rceil \right\rceil - 2\right) \geq \frac{3n}{10} - 6$$

Similarly, at least $3n/10 - 6$ elements are less than x.

Thus, in the worst case, step 5 calls Select() recursively on at most $[n - (3n/10) - 6] = \frac{7n}{10} + 6$ elements.

Time complexity analysis

- Dividing the elements into groups takes O(1)
- Computing the medians of each group takes linear time (O(n)) using insertion sort for 5 elements in each group
- Computing the median of medians of median recursively for $\lceil \frac{n}{5} \rceil$ groups takes $T(\lceil \frac{n}{5} \rceil)$
- Step 4 takes constant time i.e. O(1)
- Step 5 runs recursively and takes $T(\frac{7n}{10} + 6)$
- So the time complexity for this algorithm is

$$T(n) = T(\lceil \frac{n}{5} \rceil) + T(\frac{7n}{10} + 6) + O(n) + O(1)$$

$$= T(n/5) + T(\frac{7n}{10} + 6) + O(n)$$

To solve this recurrence relation, let's guess the solution to be O(n). [Substitution Method]

Guess T(n) = O(n)

T(n) ≤ cn

So,

$$T(n) \le c\left(\frac{n}{5}\right) + c\left(\frac{7n}{10} + 6\right) + kn$$

$$\le cn/5 + \frac{7cn}{10} + 6c) + kn$$

$$\le \frac{9cn}{10} + 6c + kn$$

$$= cn + (-cn/10 + 6c + kn)$$

Which is at most cn if (-cn/10+ 6c +kn) ≤ 0

So, T(n) ≤ cn

Therefore, T(n) = O(n)

Hence this algorithm runs in linear time.